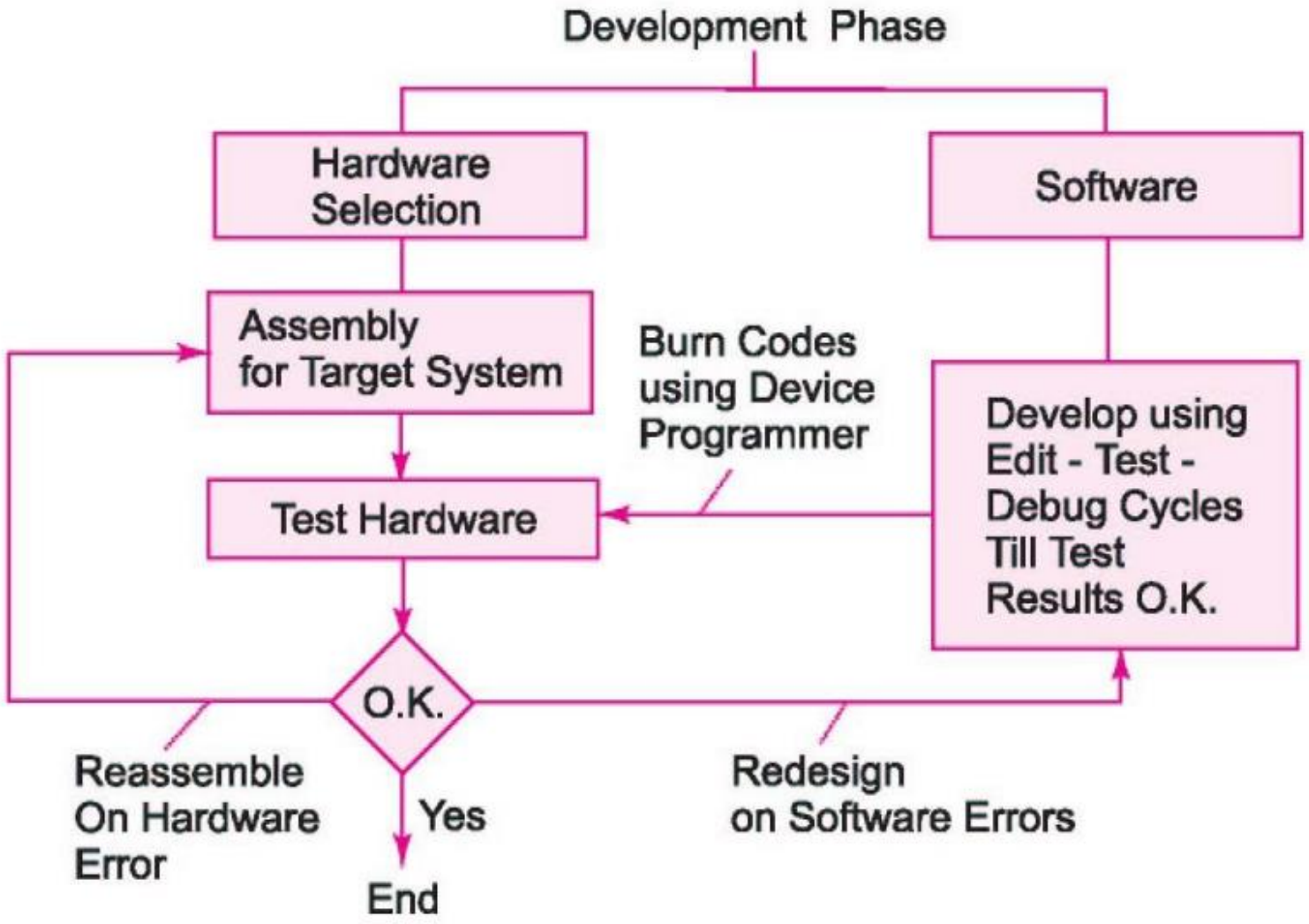
MODULE-4

Programming tools

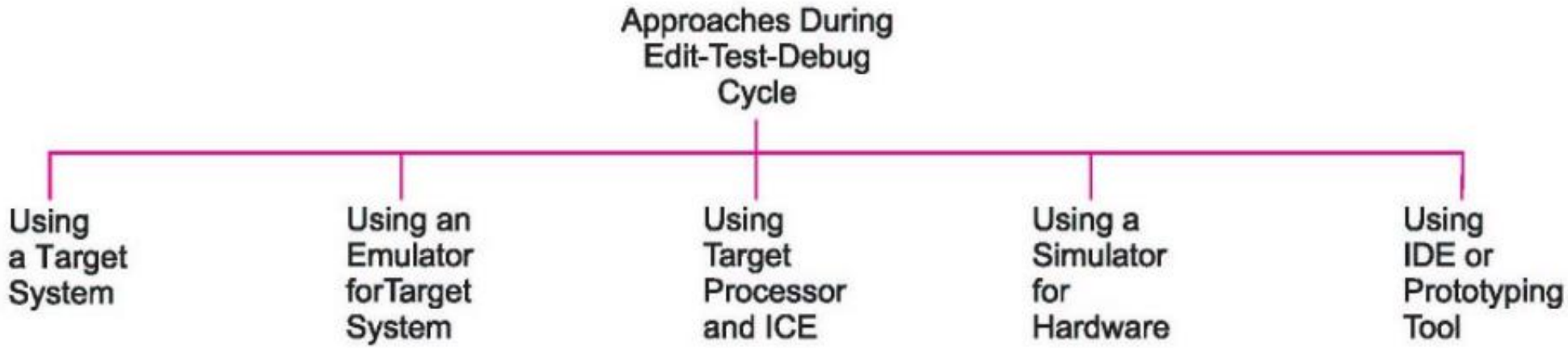
Evolution of embedded programming tools, Modelling programs, Code optimization, Logic analyzers, Programming environment

BUILDING PROCESS FOR EMBEDDED SYSTEMS

BUILDING PROCESS FOR EMBEDDED SYSTEMS



BUILDING PROCESS FOR EMBEDDED SYSTEMS



Edit-Test-Debug Cycle implementation phase of the development process

BUILDING PROCESS FOR EMBEDDED SYSTEMS

- Embedded systems programming is not substantially different from the programming you've done before.
- The only thing that has really changed is that you need to have an understanding of the target hardware platform. Furthermore, each target hardware platform is unique.
- Unfortunately, this uniqueness among hardware platforms leads to a lot of additional software complexity, and it's also the reason you'll need to be more aware of the software build process than ever before.

BUILDING PROCESS FOR EMBEDDED SYSTEMS

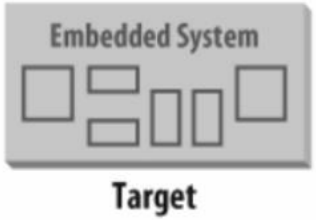
- When build tools run on the same system as the program they produce, they can make a lot of assumptions about the system.
- This is typically not the case in embedded software development, where the build tools run on a host computer that differs from the target hardware platform.
- Embedded software development tools, can rarely make assumptions about the target platform.
- Instead, the user must provide some knowledge of the system to the tools by giving them more explicit instructions.

BUILDING PROCESS FOR EMBEDDED SYSTEMS

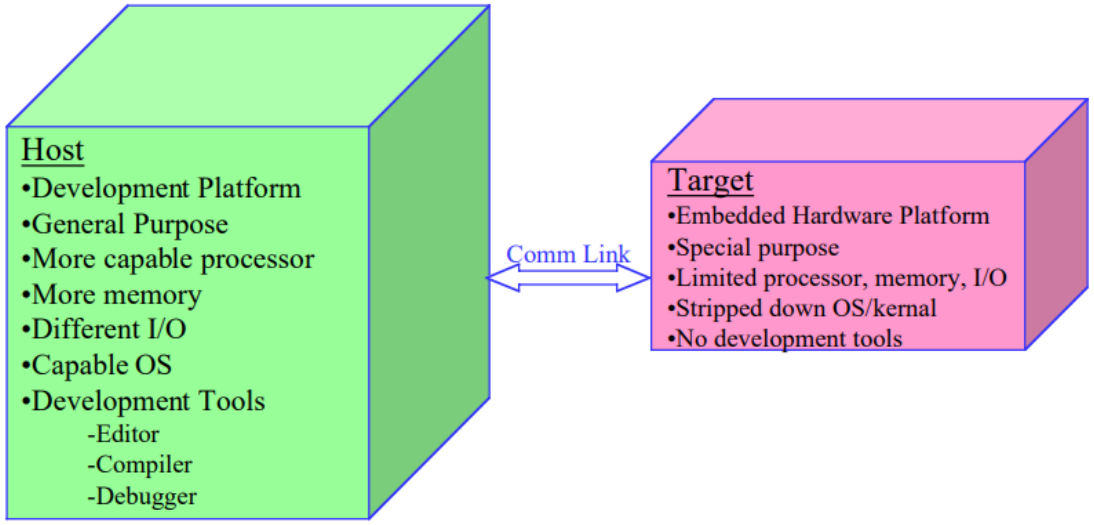
- Software Development is performed on a Host computer
  - Compiler, Assembler, Linker, Locator, Debugger
  - Produces executable binary image that will run on Target Embedded System



The development tools that build the embedded software run on a general-purpose computer



The embedded software that is built by those tools runs on the embedded system



BUILDING PROCESS FOR EMBEDDED SYSTEMS

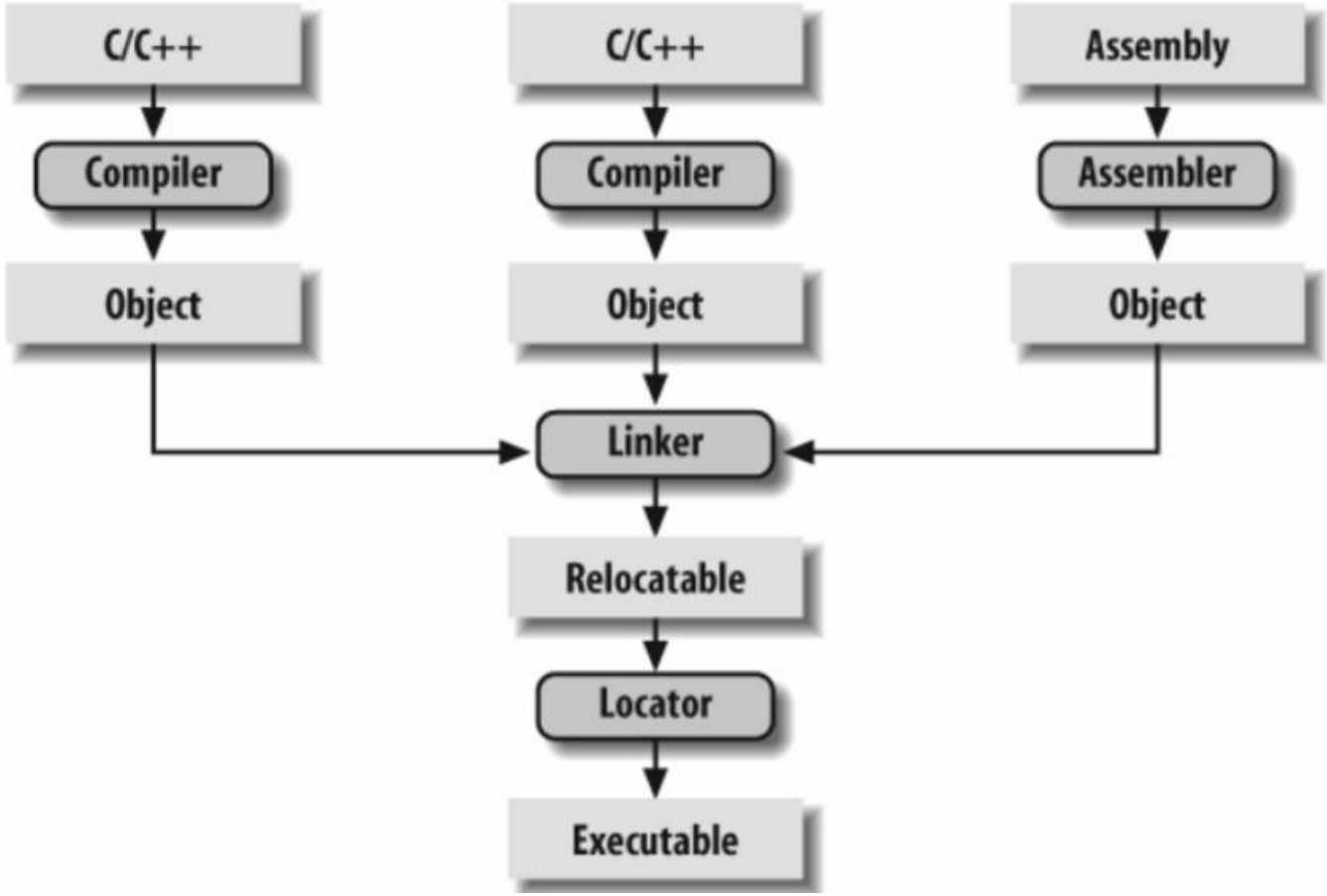
- **Software Tools**
  1. **Software Development Kit (SDK)**
  2. **Source-code Engineering Software (**Tools used to **analyze, modify, optimize, and maintain source code**.)
  3. **RTOS**
  4. **Integrated Development Environment (IDE)**
  5. **Emulator -**software that makes one system behave like another system.
  6. **Editor -** software used to write and modify text or program code.
  7. **Interpreter**(Translates and executes code line-by-line.)
  8. **Compiler-**software that translates a program from a high-level language into machine-understandable code.
  9. **Assembler-** converts Assembly Language → Machine Code
  10. **Locator (**Assigns **specific memory locations** to program sections.)
  11. **Testing and debugging tools**
  12. **Cross Assembler** An assembler that runs on a **host system** but generates code for a **different target processor**.

Example: Writing ARM code on a PC for an embedded board

PROCESS FOR DEVELOPING EMBEDDED SOFTWARE

- To develop software for a General Purpose Computer
  - Create source file
  - Type in C code
  - Build: compile and link
  - Execute: load and run
- To develop software for an embedded system
  - Create source file (on Host)
  - Type in C code (on Host)
  - Compile/Assemble: translate into machine code (on Host)
  - Link: combine all object files and libraries, resolve all symbols (on Host)
  - Locate: assign memory addresses to code and data (on Host)
  - Download: copy executable image into Target processor memory
  - Execute: reset Target processor

PROCESS FOR DEVELOPING EMBEDDED SOFTWARE



The Embedded Software Development Process

PROCESS FOR DEVELOPING EMBEDDED SOFTWARE

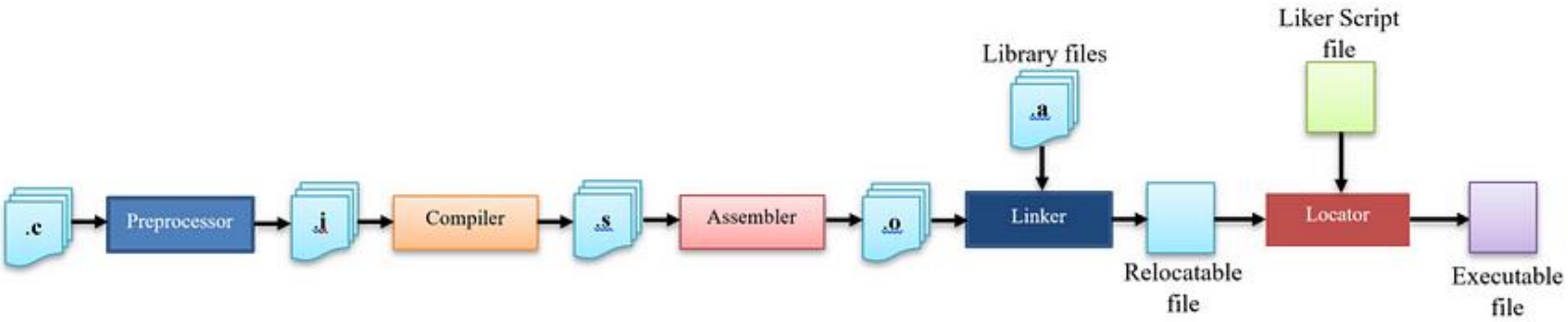
SOURCE CODE INTO EXECUTABLE BINARY IMAGE

Each of the source files must be compiled or assembled into an object file

All of the object files that result from the first step must be linked together to produce a single object file, called the re-locatable program.

Physical memory addresses must be assigned to the relative offsets within the re-locatable program in a process called relocation.

EMBEDDED SYSTEM BUILD PROCESS



EMBEDDED SYSTEM BUILD PROCESS

PREPROCESSOR

- Pre-processing is not a part of the compiler, but is a separate step in the compilation process in which all the pre-processor directives are evaluated.
- Pre-processing is a process of running a set of instructions provided by the programmer explicitly as code before the program compiles.
- The pre-processor provides the ability for the inclusion of header files, macro expansions, conditional compilation, and line control.
- Pre-processor instructions are called pre-processor directives, and they all start with a hash symbol (#). Few examples:
  - Inclusion of source files Ex: #include<stdio.h>, #include "file“
  - Macro expansion Ex: #define MIN 234
  - Conditional compilation Ex: #ifdef DEBUG printf("debugging...") #endif

EMBEDDED SYSTEM BUILD PROCESS

COMPILER

- Compiler translate programs written in high-level programming language (C or C++) to a low-level language (assembly code) for a particular processor.
- This conversion is not a one to one mapping of lines but instead a decomposition of C operations into numerous assembly operations.
- Each processor has its own unique machine language, so you need to choose a compiler that produces programs for your specific target processor.
- Types of Compiler:
  - Native-compiler: Runs on a computer platform (x86) and produces code for same computer platform (x86)
  - **Cross-Compiler:** Runs on one computer platform (x86) and produces code for different architecture (ARM)
- The use of a cross-compiler is one of the defining features of embedded software development.

EMBEDDED SYSTEM BUILD PROCESS

COMPILER

- A compiler operates in logically interrelated phases whereby source code is translated assembly file. List of phases includes,
  - Lexical Analysis: High level program is converted into a sequence of tokens. A token may be a keyword (while'), an operator (*), an identifier (variable name), literal (10 or "my string"), comment.
  - Syntax Analysis: Ensures that tokens are organised in the correct way, according to the rules of the language otherwise compiler will produce a syntax error.
  - Semantic analysis: Checks on the logical structure of the program to detect potential problems such as unused variables, uninitialized variables, etc.
  - Intermediate code generation: Intermediate representation of the final machine code is produced.
  - Code optimization: Code is optimized in order to run faster and efficiently by performing inline expansion of functions, dead code removal, loop unrolling, register allocation, etc.
  - Code generation: The optimized intermediate code is translated into assembly language for a specified machine architecture.
  - Error handling: Errors are detect and reported to enable a programmer to debug a source program.

EMBEDDED SYSTEM BUILD PROCESS

COMPILER – Memory Allocation

- After the code generation is finished, the compiler allocates memory for code and data in sections; each section has different information.

- Different memory segments are,
  - .code / .text - opcodes, Literals
  - .data - initialized global variables (and their initial values),
  - .bss - uninitialized global variables into this section.
  - . const / .rodata - constants
  - .stack / R# - local/Automatic variables (variables are defined within functions)
  - .heap - dynamic objects

- Stack and Heap segments are not allocated by the compiler but by the Linker at link-time.

EMBEDDED SYSTEM BUILD PROCESS

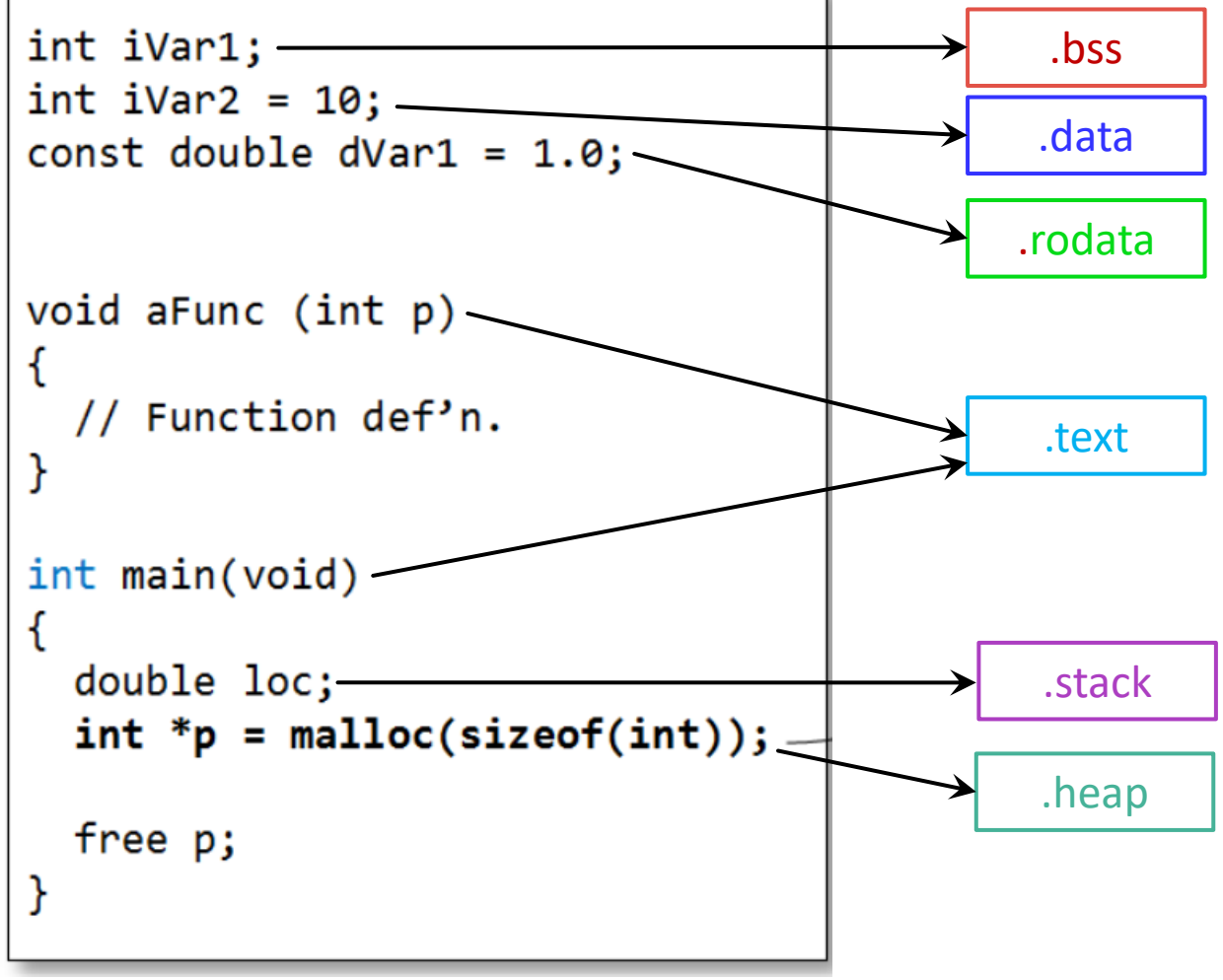
COMPILER – Memory Allocation Example

```
int iVar1;
int iVar2 = 10;
const double dVar1 = 1.0;


void aFunc (int p)
{
  // Function def'n.
}

int main(void)
{
  double loc;
  int *p = malloc(sizeof(int));

  free p;
}
```



EMBEDDED SYSTEM BUILD PROCESS

ASSEMBLER

- Assembler converts the assembly language code into an object file which is an binary file that contains set of machine-language instructions (opcodes) and data sections.
- In short, the assembler generates a microcontroller architecture-specific machine code without absolute addresses.
- Each processor has its own unique machine language, so you need to choose a appropriate assembler that produces programs for your specific target processor.
- **Cross-Assembler** is a type of assembler that is designed to generate machine code for a different processor from the one on which the assembler itself runs.
- Note that compilers nowadays can generate an object code without the need of an independent assembler.

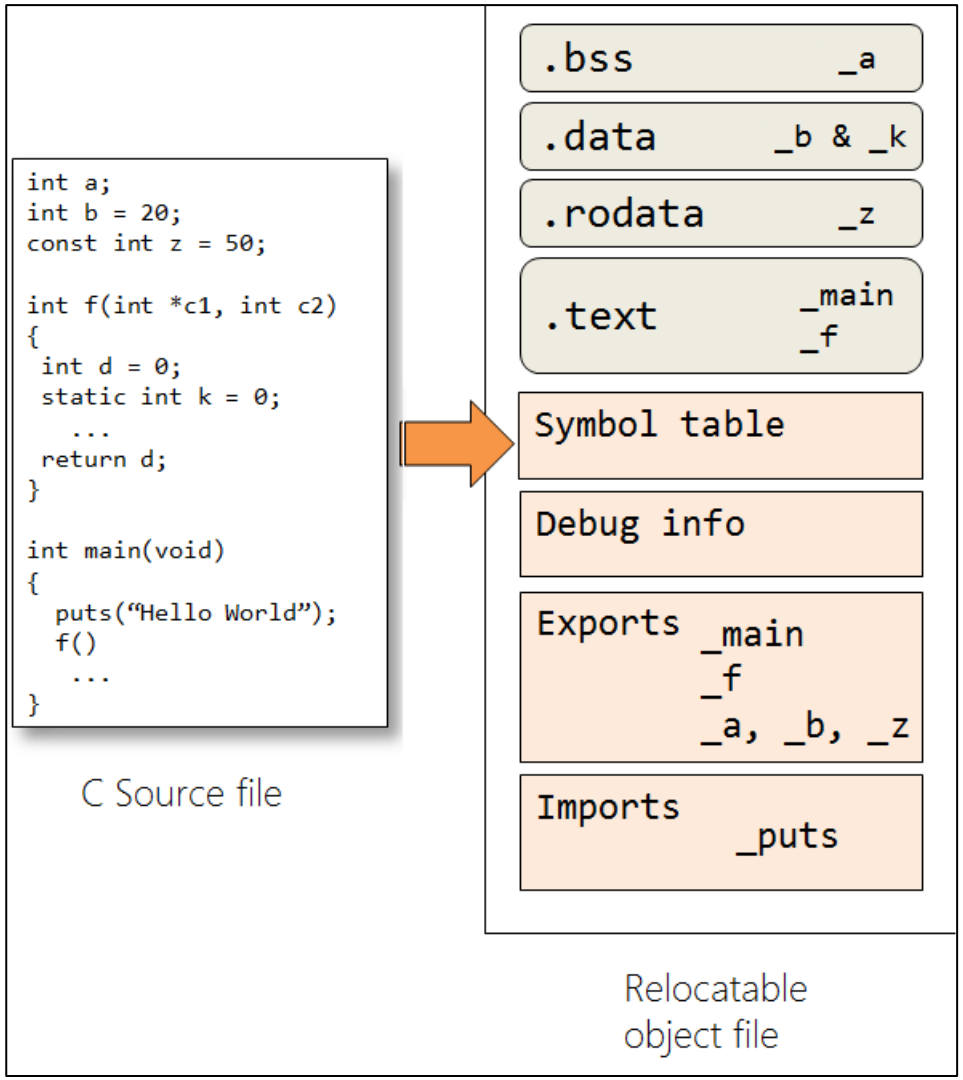
EMBEDDED SYSTEM BUILD PROCESS

ASSEMBLER

- Your embedded application may have reference to built-in libraries and more than one source code file therefore, the assembler may generate multiple object files.
- Note that the object file only contains the sections for static variables. At this stage, section locations are not fixed.
- Each object file contains following sections,
  - Symbol table - to store all variable names and their attributes.
  - Debug info section - mapping between the source code and the information needed by the debugger.
  - Exports section - contains global symbols either functions or variables.
  - Imports section - contains symbol names that are needed from other object files.
- Exports, imports, and symbol table sections are used by the linker during linking stage.

EMBEDDED SYSTEM BUILD PROCESS

ASSEMBLER



EMBEDDED SYSTEM BUILD PROCESS

LINKER

- The role of the linker is to combine multiple object files into a single relocatable file and resolve references between these multiple object files using symbols and references table.

- While combining the object files together, the linker performs the following operations:

1. Symbol resolution:
   - In multi-file program, if there are any references to symbol defined in another file, the assembler marks these references as "unresolved".
   - The linker determines the values of these references from other object files or all specified library/archive (.a) files, and patches the code with the correct values.
   - If, the Linker still cannot resolve a symbol it will report an 'unresolved reference' error.
   - If the linker finds same symbol defined in two object files, it will report a "redefinition" error.

EMBEDDED SYSTEM BUILD PROCESS

LINKER

2. Relocation: Process of changing addresses already assigned to symbols. This will also involve patching up all references to reflect the newly assigned address. Relocation process involves following,

   a) Section Merging: The Linker then concatenates like-named sections from all the input object files. The combined sections (output sections) are usually given the same names as their input sections. Program addresses are adjusted to take account of the concatenation.

   b) Section placement: When a program is assembled each section is assumed to start from address 0. And thus, symbols are assigned values relative to the start of the section. When the final executable is created, the section is placed at some address X. And all references to the symbols defined within the section, are incremented by X, so that they point to the new location.

   c) Data initialisation: Create a shadow data section (.sdata) to copy read-only sections (.code) to RAM to speed up execution

- After merging all of the code and data sections and resolving all of the symbol references, the linker produces a special "relocatable" copy of the program except no memory addresses have yet been assigned to the code and data sections within.

EMBEDDED SYSTEM BUILD PROCESS

LOCATOR

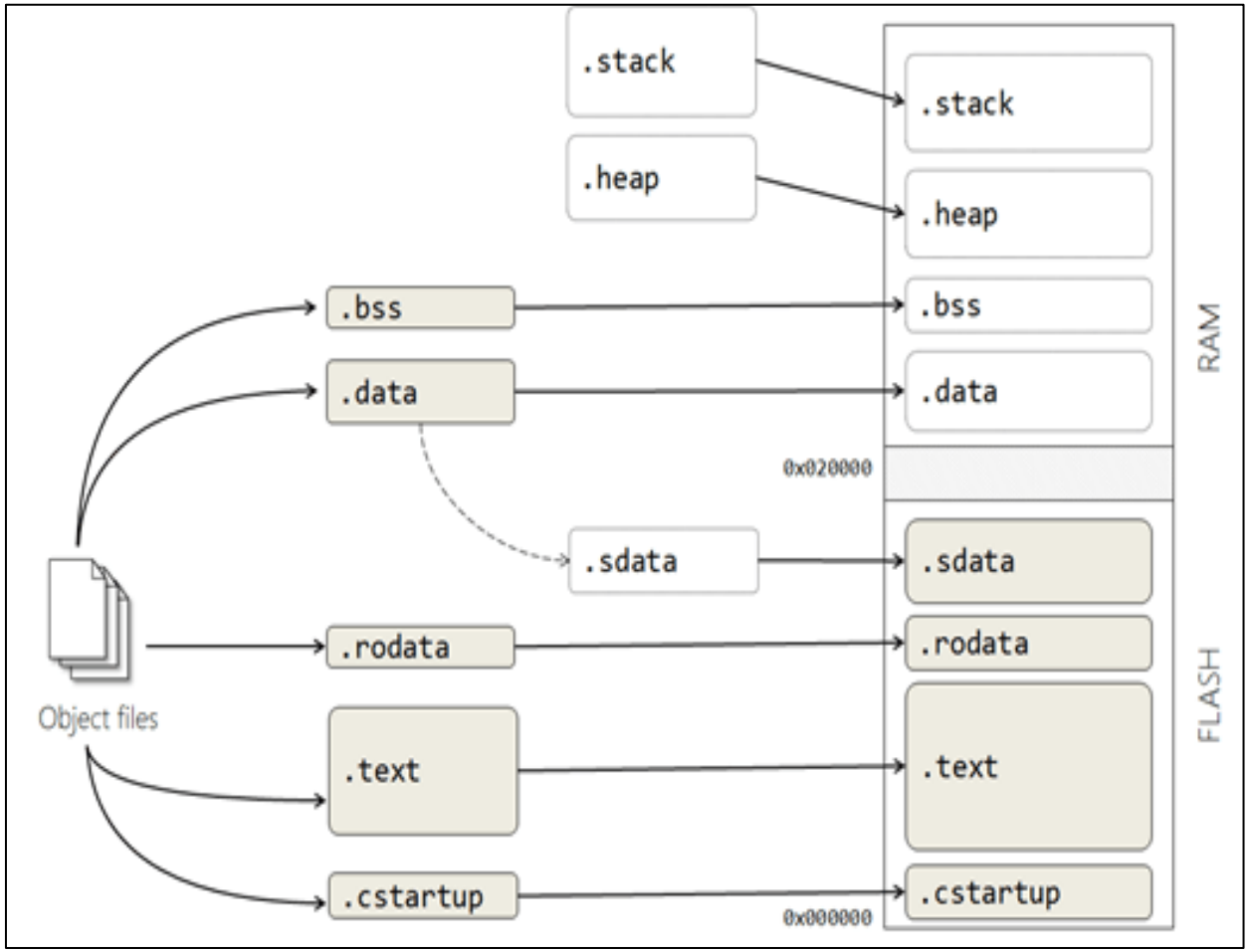
- A locator performs the conversion from relocatable program to executable binary image.
- The locator gets the output file of the linker and maps the code and data into microcontroller memory by assigning physical addresses to code and data.
- The locator will perform checks to ensure that your code and data sections will fit into the designated regions of memory.
- Linker Control File (LCF) also another name for this file is linker-script file, linker configuration file is responsible to inform the locator how to map the executable into proper addresses.
- LCF defines physical memory layout (Flash/SRAM) of the target board and placement of the different program regions to produce a single executable binary file.

EMBEDDED SYSTEM BUILD PROCESS

LOCATOR

- The intermediate output from the locating process is a load file in a platform-independent format, commonly .ELF or .DWARF (although there are many others).
- The ELF file is also used by the debugger when performing source-code debugging.
- In order to be loaded onto the target the ELF file must be converted into a native flash / PROM format (typically, .bin or .hex)
- The output of this final step of the build process is a binary image containing physical addresses for the specific embedded system.
- This executable binary image can be downloaded to the embedded system or programmed into a memory chip.

EMBEDDED SYSTEM BUILD PROCESS



Mapping different sections into microcontroller memory

- .cstartup – the system boot code – is explicitly located at the start of Flash.
- .text and .rodata are located in Flash, since they need to be persistent
- .stack and .heap are located in RAM.
- .bss is located in RAM in this case but is (probably) empty at this point. It will be initialised to zero at start-up.
- .data section is located in RAM (for run-time) but its initialisation section, .sdata, is in ROM.

EMBEDDED SYSTEM BUILD PROCESS

RUN-TIME LIBRARY

- A runtime library is a collection of software routines and functions that provide low-level support for a programming language during the execution of a program.
- It consists of precompiled code that can be linked to a program at either compile time or runtime.
- When the library files remain out of compilation of the main executable and comes into use only when required at runtime, it is called dynamic linking.
- These library files can be custom programmed or provided by the language compiler for standard functions. e.g. C compilers provide C-Standard runtime libraries
- For this reason, some programming bugs are not discovered until the program is tested in a "live" environment – Runtime error

EMBEDDED SYSTEM BUILD PROCESS

MAKEFILE

- Manually entering individual compiler and linker commands on the command line becomes tiresome very quickly. In order to avoid this, a makefile can be used.
- A Makefile is a script file used in software development projects to automate the building and compilation of source code into executable programs or libraries.
- Makefiles are associated with the "make" build automation tool, which reads the instructions from the Makefile and executes the necessary commands to build the project.
- The make utility follows the rules in the makefile in order to automatically generate output files from a set of input source files.
- Makefiles are commonly used in Unix and Unix-like systems, but they can also be used on other platforms with tools like GNU Make.

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

- In this section, we show an example **build procedure for the Arcom VIPER-Lite development board.**
- If another hardware platform is used, a **similar process should be followed** using the tools and conventions that accompany that hardware.
- Once the tools are installed, the commands covered in the following sections are entered into a **command shell.**
- For Windows users, the command shell is a **Cygwin bash shell** (Cygwin is a Unix environment for Windows); for Linux users, it is a regular **command shell.**

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

COMPILE

- First we look at the individual commands in order to manually perform the three separate tasks (compiling, linking, and locating)
- Later, we will learn how to automate the build procedure with makefiles.
- The Blinking LED example consists of two source modules: *led.c and blink.c.*
- The first step in the build process is to compile these two files.

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

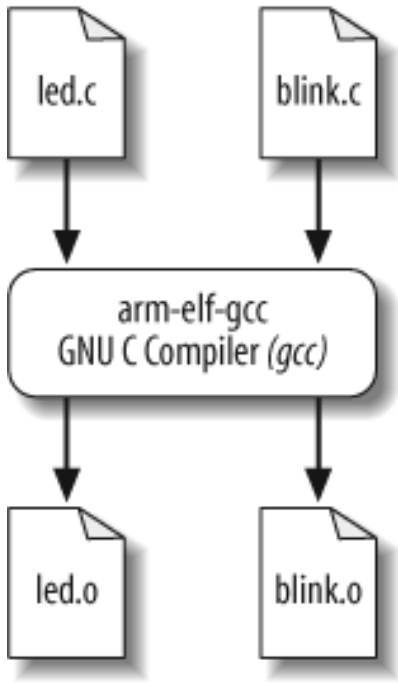
COMPILE

```
arm-elf-gcc [
                options
            ]
                file
            ...
```

- The basic structure for the gcc compiler command is:

- The command-line options we'll need are:

-g	To generate debugging info in default format
-c	To compile and assemble but not link
-Wall	To enable most warning messages
-I../include	To look in the directory include for header files



- Here are the actual commands for compiling:

```
# arm-elf-gcc -g -c -Wall -I../include led.c
# arm-elf-gcc -g -c -Wall -I../include blink.c
```

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

COMPILE

- We broke up the compilation step into two separate commands, but you can compile the two files with one command.
- To use a single command, just put both of the source files after the options. If you wanted different options for one of the source files, you would need to compile it separately as just shown.
- The result of each of these commands is the creation of an object file that has the same prefix as the .c file, and the extension .o.
- So if all goes well, there will now be two additional files—led.o and blink.o—in the working directory.

LINK AND LOCATE

- We now have the two object files—led.o and blink.o—that we need in order to perform the second step in the build process.
- As we discussed earlier, the GNU linker performs the linking and locating of the object files.
- For the third step, locating, there is a linker script file named viperlite.ld that we input to ld in order to establish the location of each section in the Arcom board's memory.

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

LINK AND LOCATE

```
arm-elf-ld [
                options
           ]
                file
           ...
```

- The structure for the linker and locater ld command is:
- The command-line options we'll need for this step are:

-Map blink.map	To generate a map file and use the given filename
-T viperlite.ld	To read the linker script
-N	To set the text and data sections to be readable and writable
-o blink.exe	To set the output filename (if this option is not included, ld will use the default output filename a.out)

- The actual command for linking and locating is:

```
# arm-elf-ld -Map blink.map -T viperlite.ld -N -o blink.exe led.o blink.o
```

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

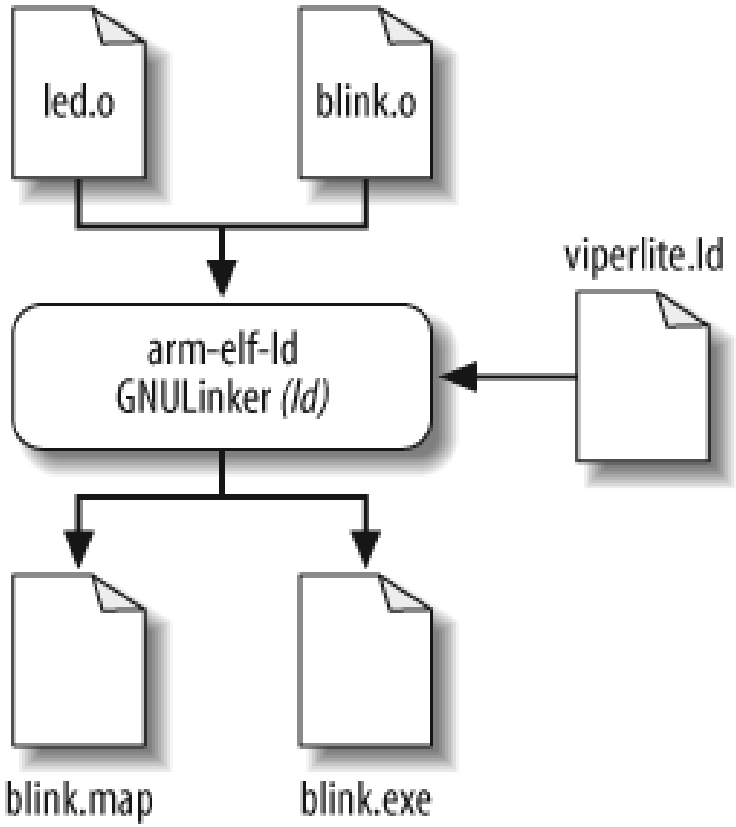
LINK AND LOCATE

- The order of the object files determines their placement in memory. Because we are not linking in any startup code, the order of the object files is irrelevant.
- If startup code were included, you would want that object file to be located at the proper address.
- The linker script file can be used to specify where you want the startup routine (and other code) to reside in memory.
- Furthermore, you can also use the linker script file to specify exact addresses for code or data, should you find it necessary to do so.

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

LINK AND LOCATE

- As you can see in this command, the **two object files—led.o and blink.o—**are the last arguments on the command line for linking.
- The linker script file, **viperlite.ld**, is also passed in for locating the data and code in the Arcom board's memory.
- The result of this command is the creation of **two files—blink.map and blink.exe—**in the working directory.



LINK AND LOCATE

- The .map file gives a complete listing of all code and data addresses for the final software image.
- It provides information similar to the contents of the linker script described earlier.
- However, these are results rather than instructions and therefore include the actual lengths of the sections and the names and locations of the public symbols found in the relocatable program.

LINKER MAP FILES

- The linker map file provides valuable information that can help you understand and optimize memory.
- The map file is a symbol table for the whole program.
- The most straightforward pieces of information in the map file are the actual memory regions, with location, size and access rights granted to those regions

```
Memory Configuration

Name             Origin             Length             Attributes
FLASH            0x0000000000001000 0x00000000000ff000 xr
RAM              0x0000000020000008 0x0000000000003fff8 xrw
*default*        0x0000000000000000 0xffffffffffffffff
```

LINKER MAP FILES

- **Linker script and memory map section** contains a breakdown of the memory contribution of each and every file that was linked into the final image.

```
Linker script and memory map

.text           0x0000000000001000      0x8c8
 ***(.isr_vector)**
 .isr_vector    0x0000000000001000      0x200 _build/nrf52840_xxaa/**gcc_startup_nrf52840.
                0x0000000000001000                __isr_vector
 *(.text*)
 .text          0x0000000000001200       0x40 /usr/local/Cellar/arm-none-eabi-gcc/8-2018-q
 .text          0x0000000000001240       0x74 /usr/local/Cellar/arm-none-eabi-gcc/8-2018-q
                0x0000000000001240                _mainCRTStartup
                0x0000000000001240                _start
 .text          0x00000000000012b4       0x3c _build/nrf52840_xxaa/gcc_startup_nrf52840.S.
                0x00000000000012b4                Reset_Handler
                0x00000000000012dc                NMI_Handler

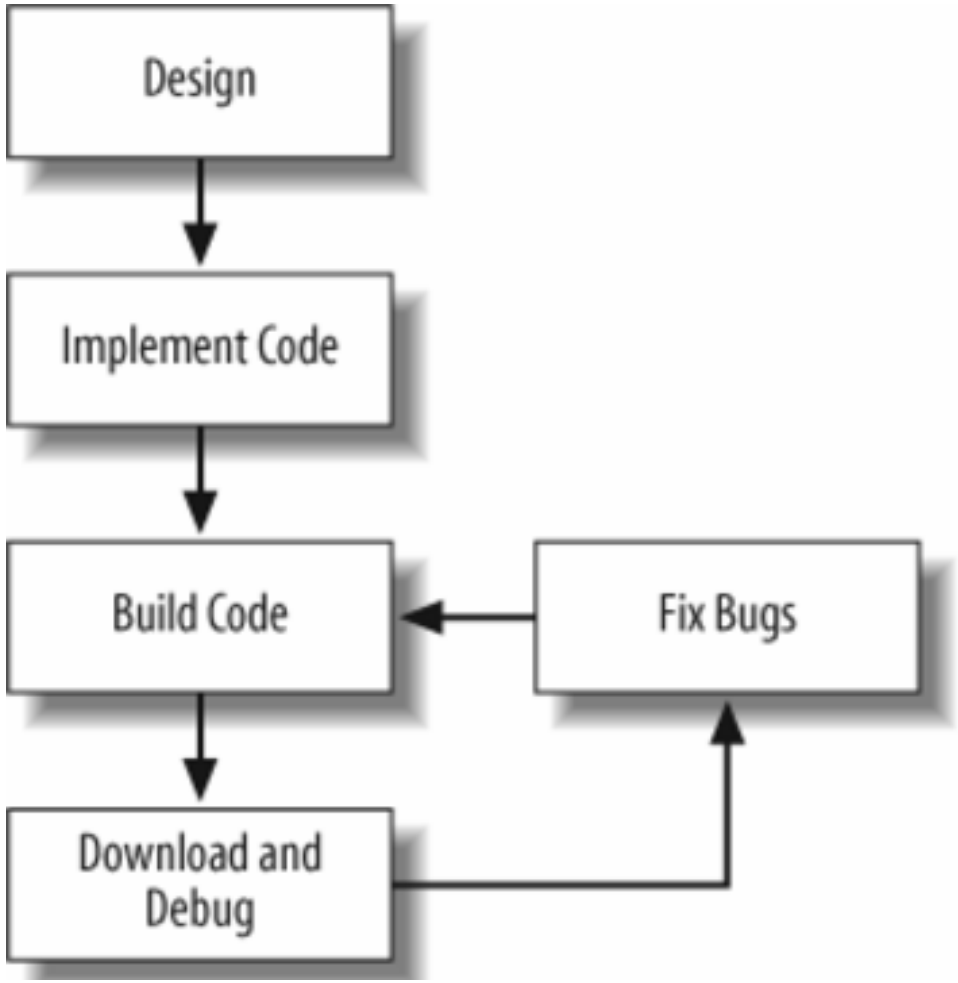
[...]

 .text.**bsp_board_led_invert**
                0x00000000000012f0       **0x34** _build/nrf52840_xxaa/**boards.c.o**
                0x00000000000012f0                bsp_board_led_invert
```

LOADING ON THE TARGET

- Previously, we saw how the code or **software to be executed on the embedded system (target board) is written on a computer.**
- The resulting code created after subjecting it to be build process is called the ***binary executable image or simply hex code.***
- This topic explains how the ***hex code*** is loaded on the target board which is referred as **downloading**
- And what are the various possible ways of **debugging a code** meant to run on a embedded system.

LOADING ON THE TARGET



- There are two ways of downloading the binary image on the embedded system:

1. Using a Device Programmer
2. In-system programming (ISP)

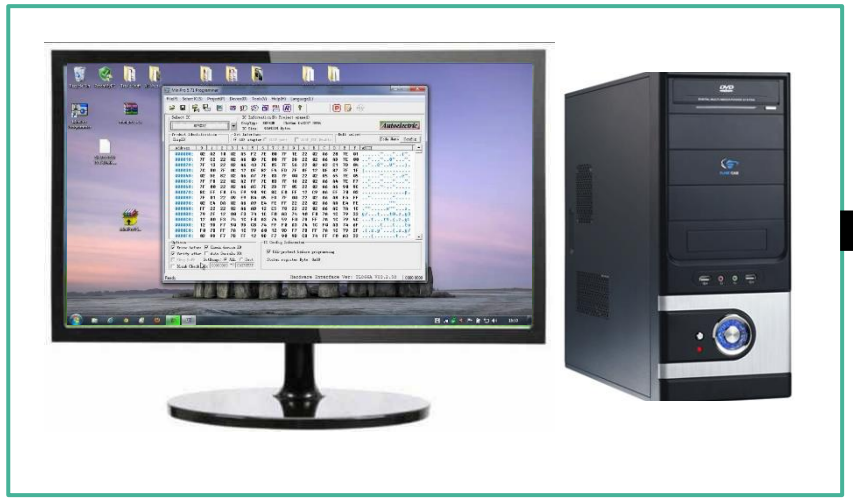
LOADING ON THE TARGET

1. **Using a Device Programmer:**

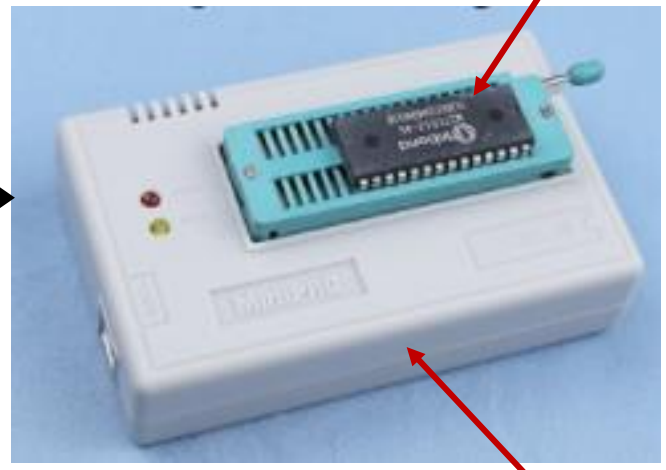
   - Step 1: Once the binary image is ready on the computer, the device programmer is connected to the computer.
   - Step 2: The uP/uC or memory chip, usually the ROM which is supposed to contain the binary image is placed on the proper socket on the device programmer.
   - Step 3: The device programmer contains a software interface through which the user selects the target uP/uC for which the binary image has to be downloaded.
   - Step 4: The Device programmer then transfers the binary image bit by bit to the chip.

LOADING ON THE TARGET

1. Using a Device Programmer:



Host Computer



LOADING ON THE TARGET

2. Using In System Programmer(ISP):

- Certain Target embedded platforms uses a piece of hardware called ISP that have a hardware interface to both the computer as well the target board's chip where the code is to be downloaded.

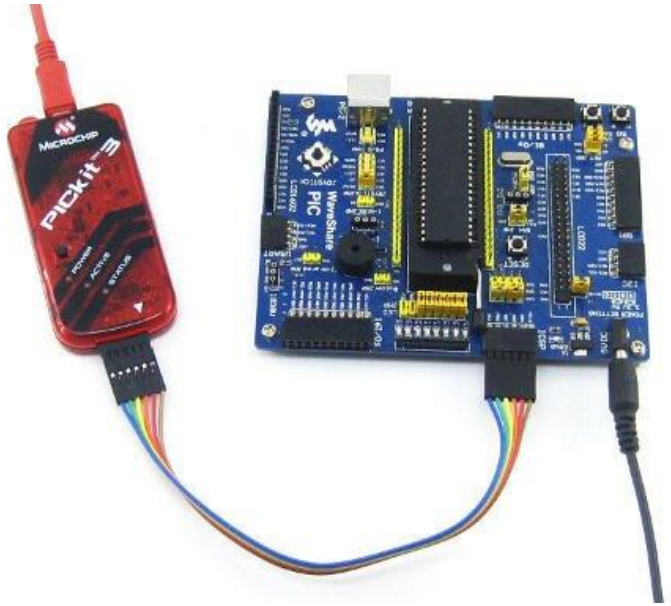
- ISP is also called in-circuit serial programming (ICSP), is the ability of programming the embedded processor/controller while it is installed in a complete system, rather than requiring the chip to be programmed prior to installing it into the system.

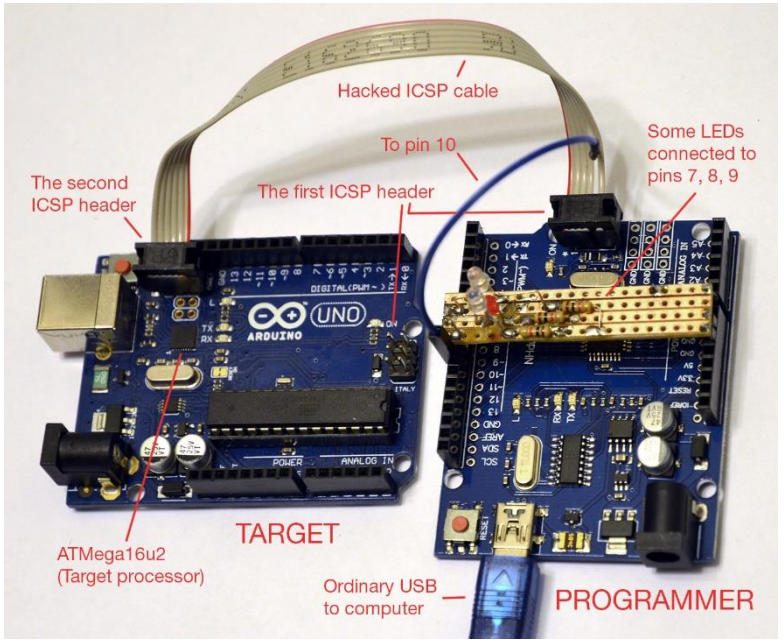
- It also allows firmware updates to be delivered to the on-chip memory of microcontrollers and related processors without requiring specialist programming circuitry on the circuit board, and simplifies design work

LOADING ON THE TARGET

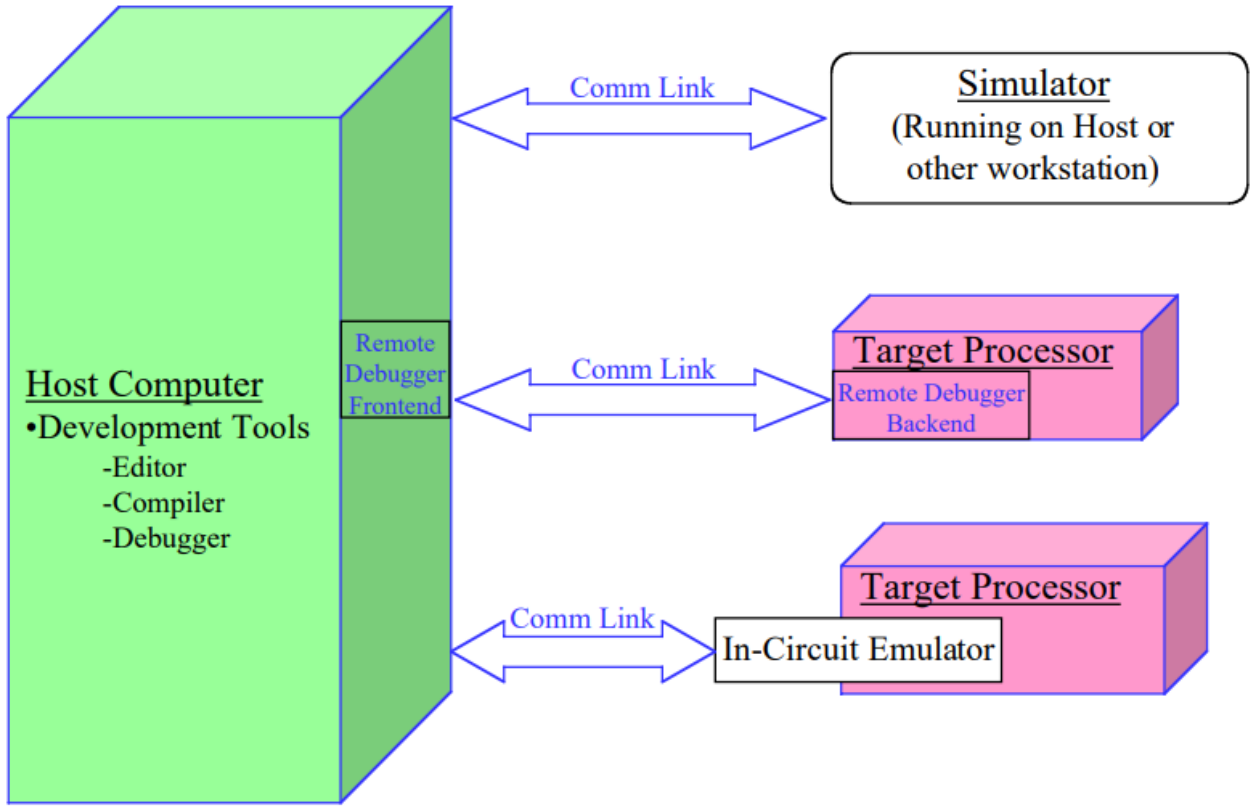
2. Using In System Programmer(ISP):

- The user through the ISP's software interface sends the binary image to the target board. This avoids the requirement of frequently removing the microprocessor / microcontroller or ROM for downloading the code if a device programmer had to be used.





DEBUGGING TOOLS



DEBUGGING TOOLS

SIMULATOR

- Simulator is a host-based program that simulates functionality and instruction set of target processor.
- The front-end has text or GUI-based windows for source code, register contents, etc.
- Simulators are valuable during early stages of development.
- Disadvantage of this method is that, it only simulates the processor, and not the peripherals.

DEBUGGING TOOLS

REMOTE DEBUGGER

- Remote debuggers are one of the commonly used downloading and testing tools during development of embedded software
- It is used to monitor/control embedded SW. It is used to download, execute and debug embedded software over a comm. link.
- The program running on the host of a remote debugger has a user interface (GUI/Command-line) that looks just like other debugger
- The front-end of the GUI debuggers contain several windows to show the active part of the source code, current register contents, and other relevant information about the executing program.

DEBUGGING TOOLS

REMOTE DEBUGGER

- Backend provides low-level control of target processor, runs on target processor and communicates to the front-end over a communication link.
- Debugger and software being debugged are executing on two different computer systems.

  -Start/restart/kill, and stepping through program.

  - Software breakpoints.

  - Reading/writing registers or data at specified address.
- Disadvantage: inability to debug startup Code, code must execute from RAM, requires a target processor to run the final software package

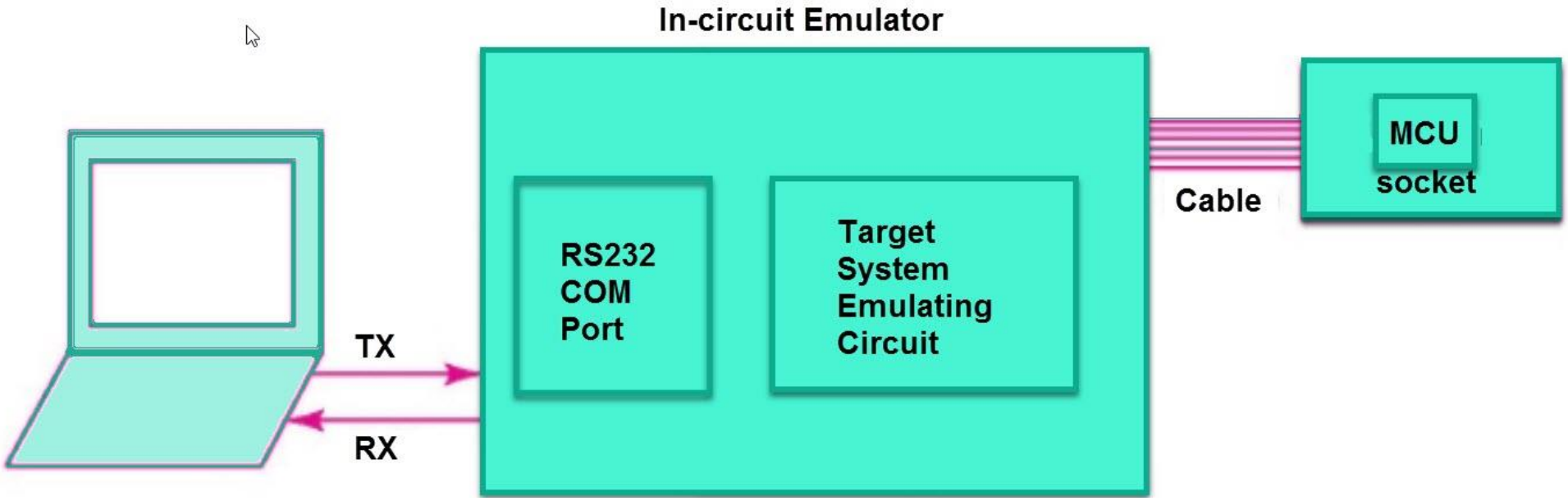
DEBUGGING TOOLS

IN-CIRCUIT EMULATOR (ICE)

- Emulation refers to the ability of a computer program or electronic device to imitate another program or device.
- An emulator is a piece of hardware/software that enables one computer system to run programs that are written for another computer system.
- An in-circuit emulator (ICE) provides a lot more functionality than a remote debugger.
- In addition to providing the features available with a remote debugger, an ICE allows you to debug startup code and programs running from ROM, etc.,

DEBUGGING TOOLS

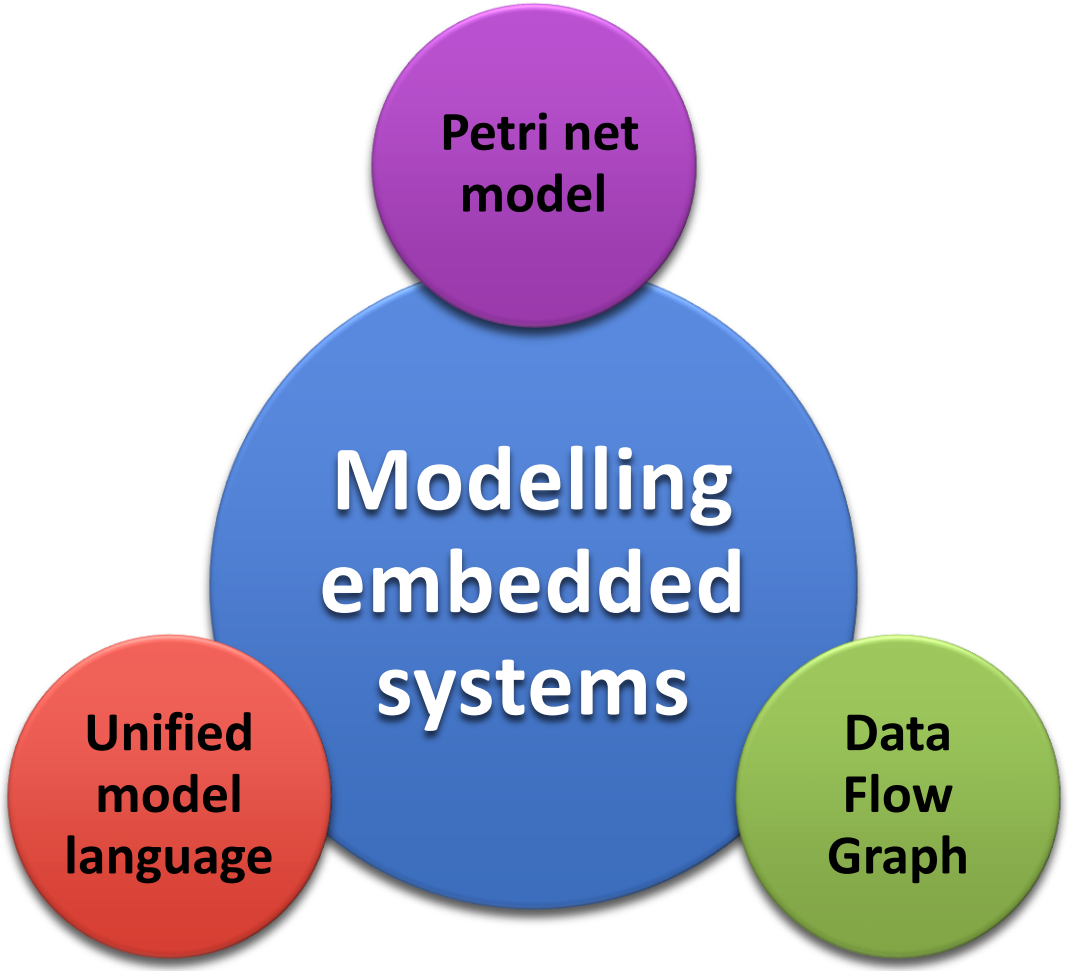
IN-CIRCUIT EMULATOR (ICE)



DEBUGGING TOOLS

IN-CIRCUIT EMULATOR (ICE)

- In-Circuit Emulator (ICE) takes the place of the target processor. It contains a copy of target processor, plus RAM, ROM, and its own embedded software.
- It allows you to examine the state of the processor while the program is running. It uses the remote debugger for human interface.
- ICE provides greater flexibility, ease for developing various applications on a single system in place of testing that multiple targeted systems. Disadvantage of this method is that, it is expensive.


Petri net model
Modelling embedded systems
Unified model language
Data Flow Graph

Modelling Programs

Representation of a program

- Modelling the programs in Embedded systems.
- Program has basic blocks, conditional statements etc.
- Can be represented by
  - Data Flow graph (DFG)
  - Control Data Flow Graph (CDFG)

Modelling Programs

Representation of a program

- Two types of nodes
- Round nodes - represents or denotes operators.
- Square nodes - represents values.

Modelling Programs

$$w = a + b;$$

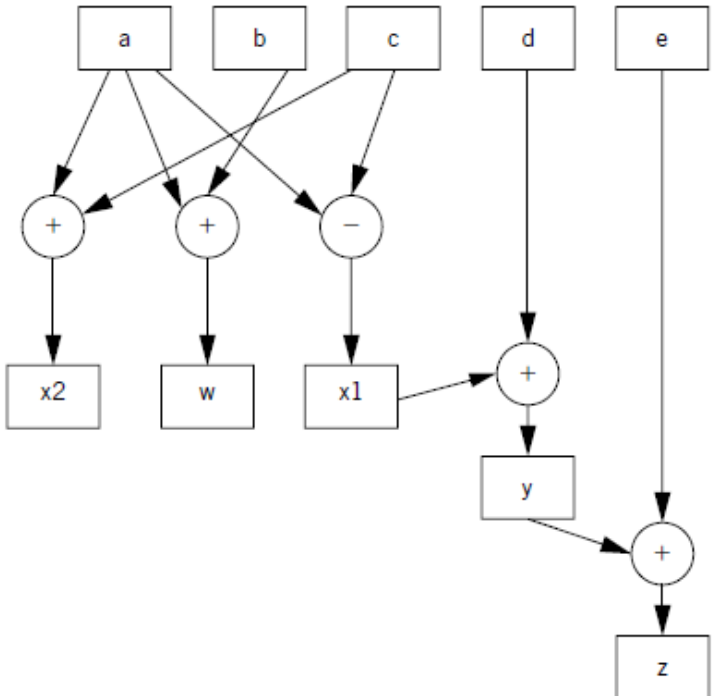
$$x1 = a - c;$$

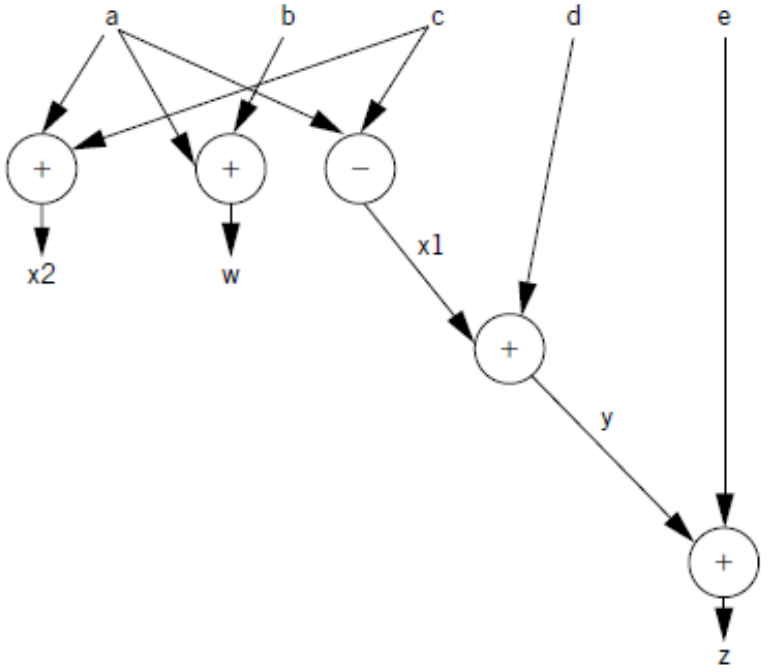
$$y = x1 + d;$$

$$x2 = a + c;$$

$$z = y + e;$$

Single assignment form





Modelling Programs

Advantages of DFG

- It orders the way operations can be performed in a program.
- Determines the feasible reordering's of the operations, this will reduce the pipeline or cache conflicts

Modelling Programs

Control DFG

- Constructs a model for decision flow and data flow of a program.
- Data flow nodes represented by basic block (set of statements).
- Control flow of a sequential program is represented by decision nodes

Modelling Programs

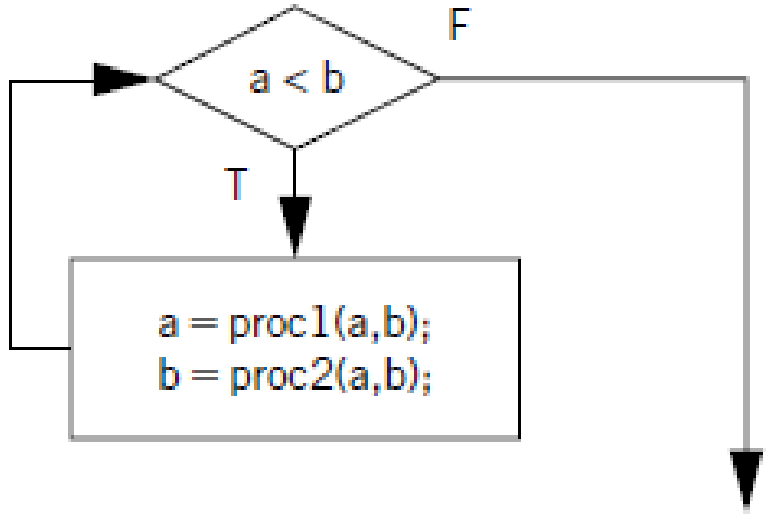
Control DFG

- Basic block nodes
- Decision nodes
- Edges labelled with possible outcomes

Modelling Programs

Building loop in CDFG

```
while (a < b)
{
a = proc1(a,b);
b= proc2(a,b);
}
```

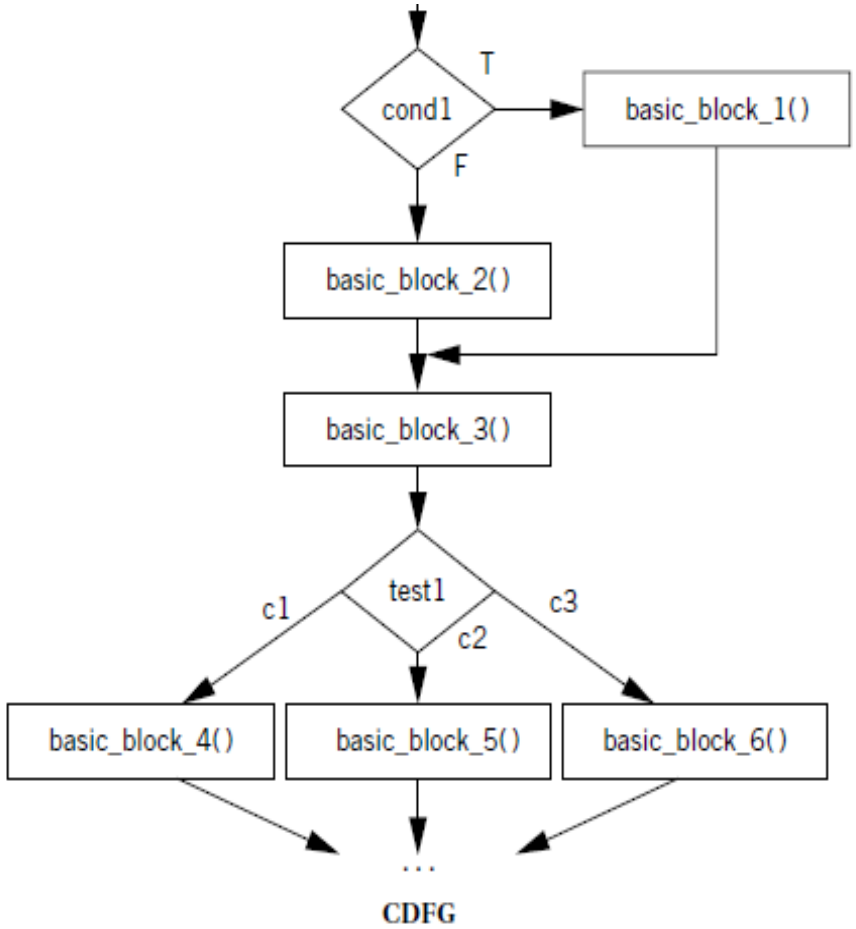


CDFG

Modelling Programs

Condition blocks using CDFG

```
if (cond1)
basic_block_1( );
else
basic_block_2();
basic_block_3( );
switch (test1) {
case c1: basic_block_4( ); break;
case c2: basic_block_5( ); break;
case c3: basic_block_6( ): break;
}
```



CDFG

Modelling Programs

Control DFG

- CDFG is a hierarchical representation.
- Each block in CDFG is expanded using Data flow graph.

PETRINET MODEL

PETRI NET MODEL

- Petri net can also called as Place/Transition net
- It is used for describing and analyzing concurrent process
- Petri net is also a graphical tool
- Petri Net(PN) is Very similar to State Transition Diagrams which model the system behavior
- It is an abstract model to show the interaction between asynchronous processes
- In asynchronous process, start and sequence of processes may vary during the execution

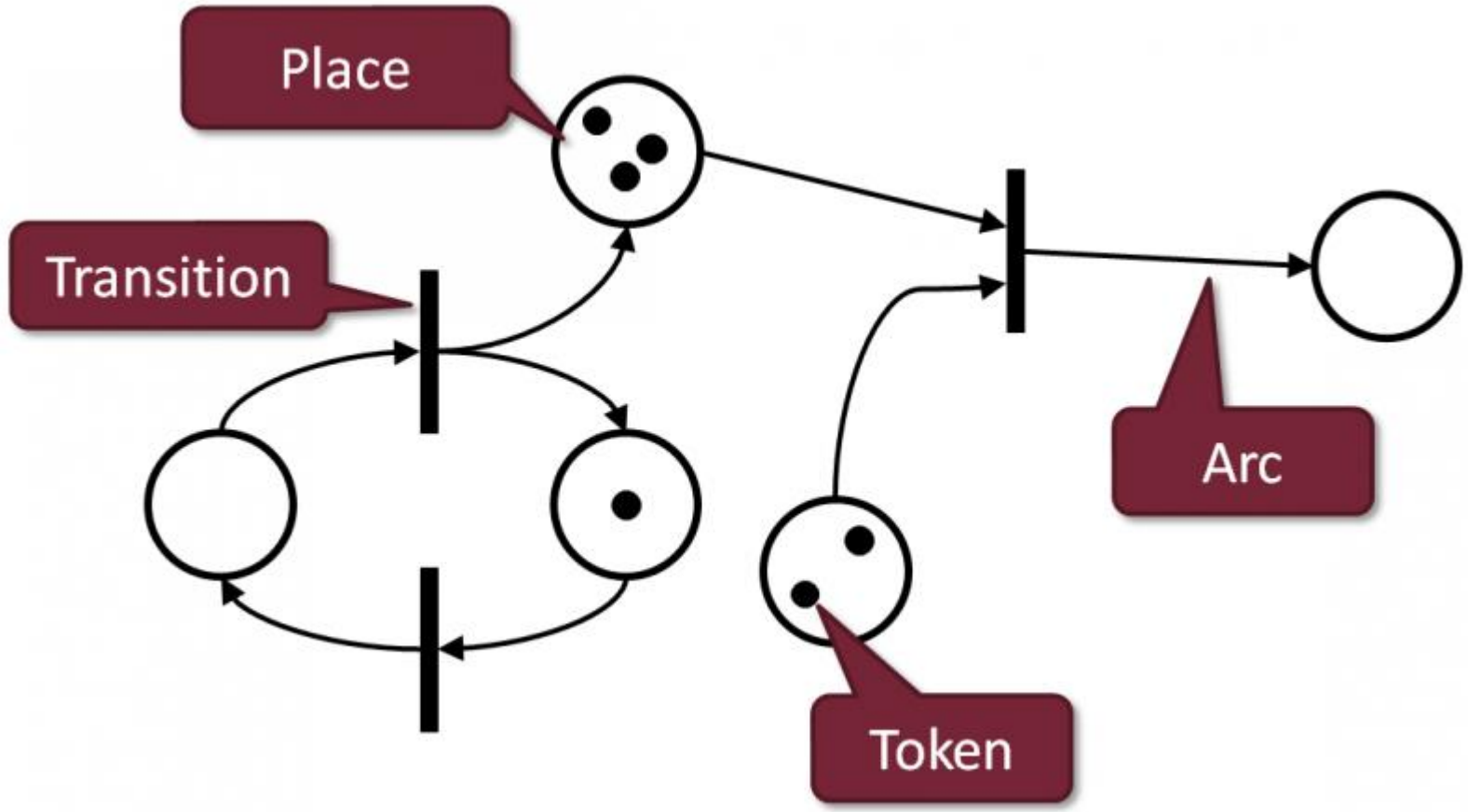
PETRINET MODEL

COMPONENTS OF PETRINET MODEL

- Petri net Model consists of four types of components - places (circles), transitions (rectangles) and arcs (arrows), tokens(black dot):
  - Places represent possible states of the system;
  - Transitions are events which cause the change of state
  - Every arc simply connects a place with a transition or a transition with a place.
  - Change of state is denoted by a movement of token(s) from place(s) to place(s)

PETRINET MODEL

COMPONENTS OF PETRINET MODEL



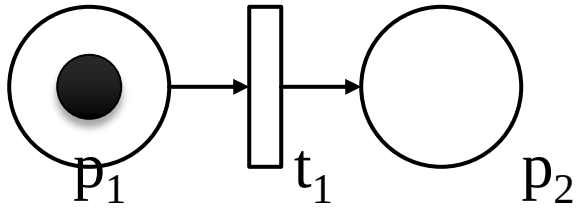
PETRINET MODEL

- Change of state is caused by the firing of a transition
- Firing refers to the occurrent of an event
- The firing is based on the input conditions, denoted by token availability
- A transition is enabled when sufficient tokens are available in input places
- Once firing is initiated, tokens will be transferred from input to output places

PETRINET MODEL

COMPONENTS OF PETRINET

- Below is an example Petri net with two places and one transaction.

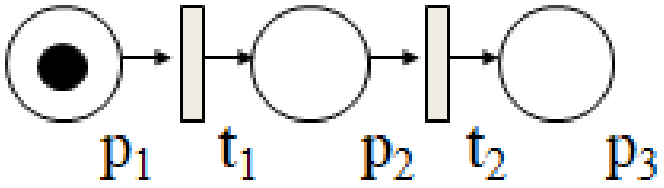


- p_1: input place
- p_2: output place

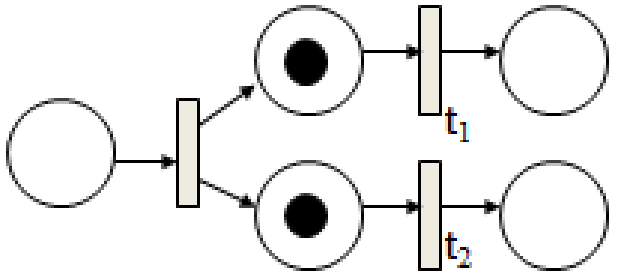
PETRINET MODEL

PROPERTIES

- **Sequential Execution:** Transition t2 can take place only after t1. Here a constraint is imposed "t2 after t1"



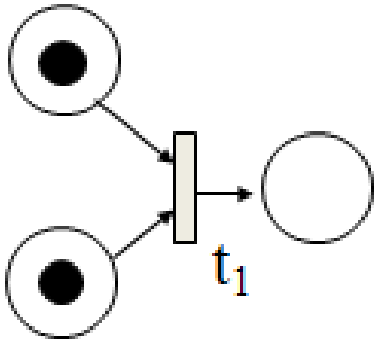
- **Concurrency:** This property is used in Modelling distributed control system. t1 and t2 are concurrent process



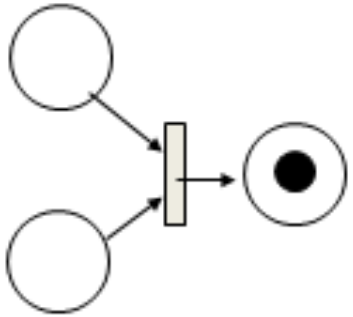
PETRINET MODEL

PROPERTIES

- **Merging:** When several tokens are arrived at same transition merging will take place



- **Synchronization:** Transition t1 will be enabled only when at least one token is available at each of its input places



PETRINET MODEL

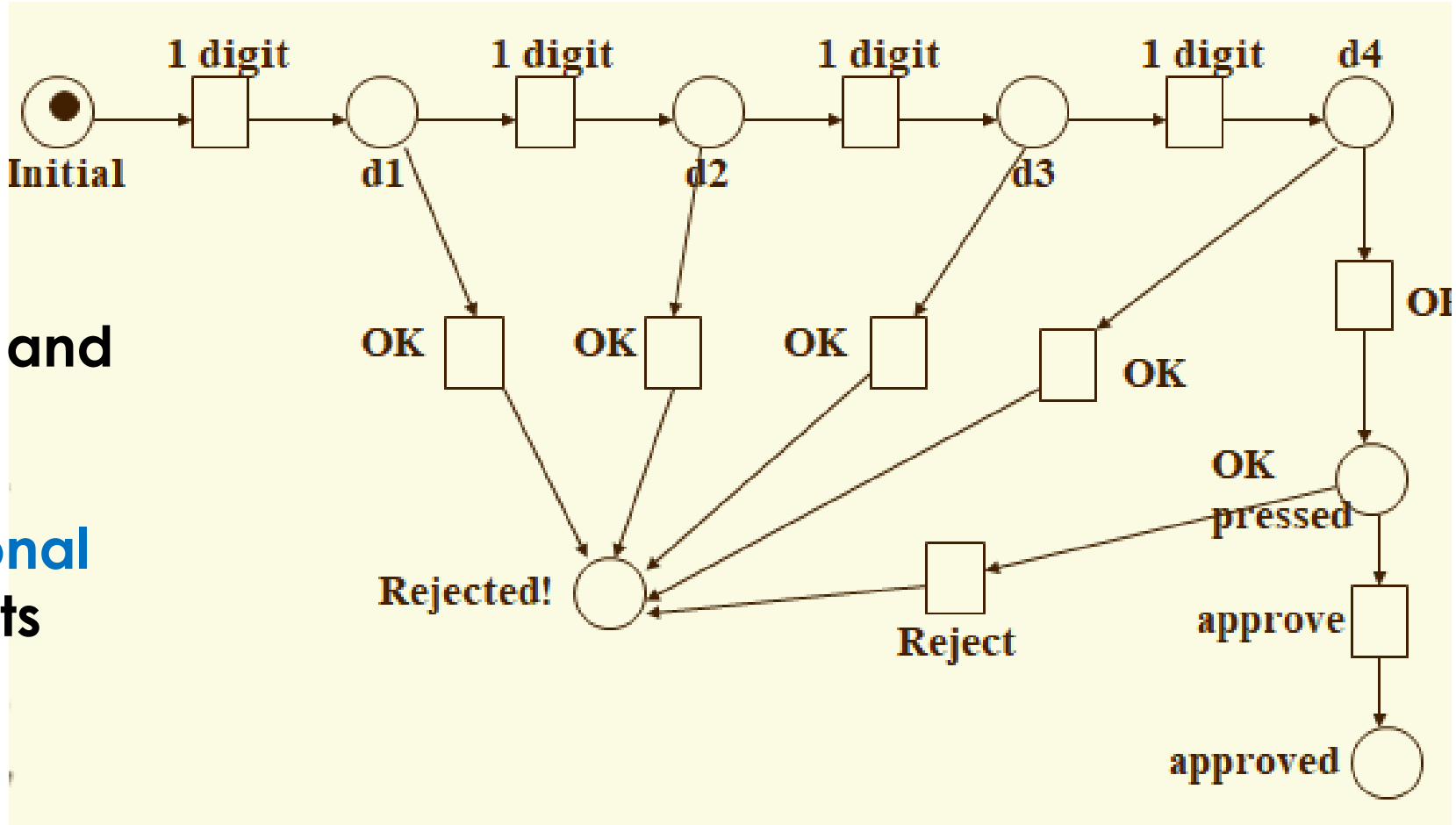
EXAMPLE-1 POINT OF SALE (POS) MACHINE

- A point of sale terminal (POS terminal) is an electronic device used to process card payments at retail locations. A POS terminal generally does the following:
  - Reads the information off a customer's credit or debit card
  - Validate the user using 4-digit PIN number
  - Checks whether the funds in a customer's bank account are sufficient
  - Transfers the funds from the customer's account to the seller's account
  - Records the transaction and prints a receipt

PETRINET MODEL

EXAMPLE-1 POINT OF SALE (POS) MACHINE

- **Scenario 1: Normal**
  - Enters all 4 digits and press OK.
- **Scenario 2: Exceptional**
  - Enters only 3 digits and press OK.



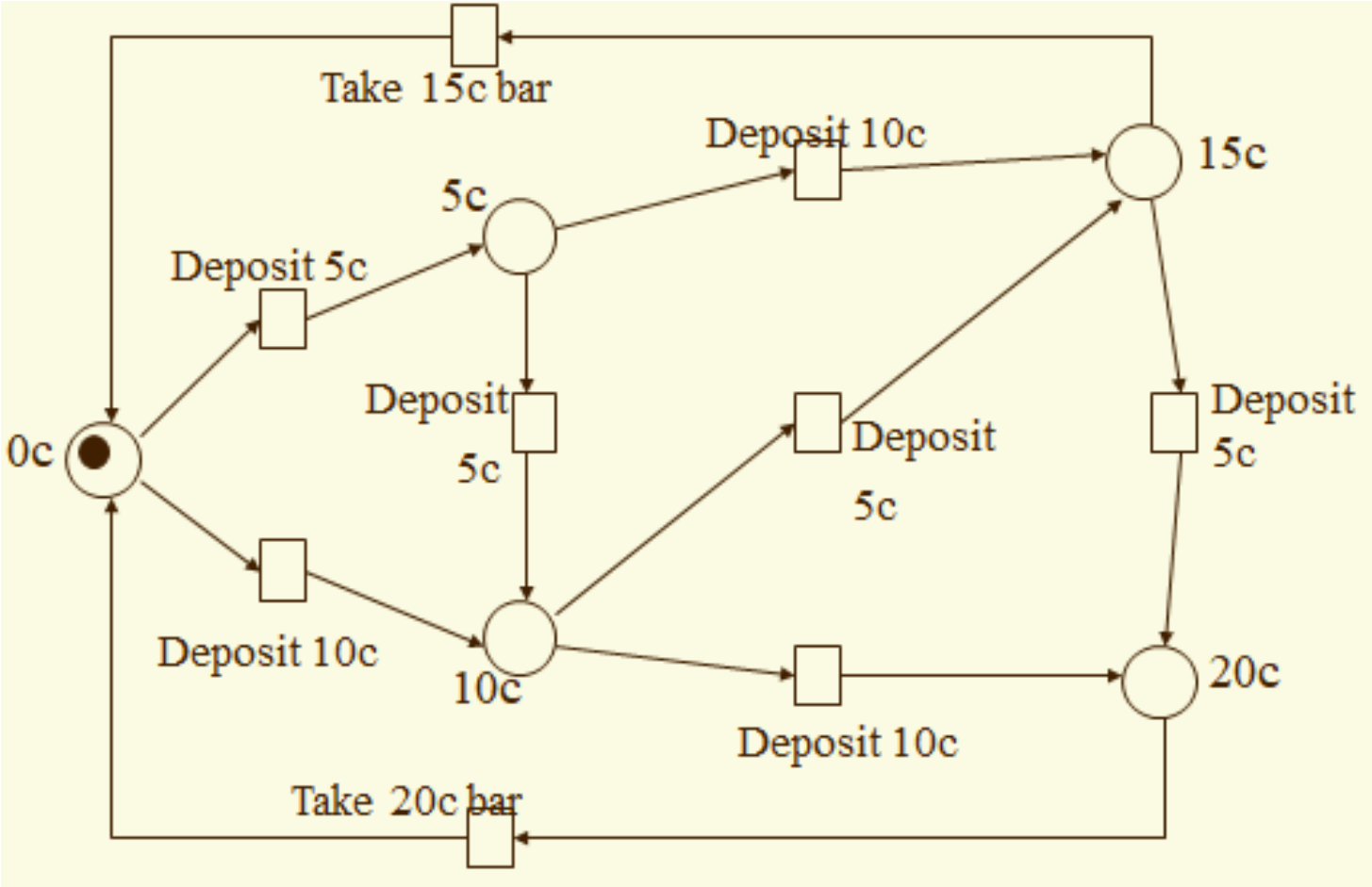
PETRINET MODEL

EXAMPLE-2 VENDING MACHINE

- The machine dispenses two kinds of snack bars 20c and 15c. Constraint: 10c and 5c coins can only be used
- Scenario 1:
  - Deposit four 5c, take 20c snack bar.
- Scenario 2:
  - Deposit 10 + 5c, take 15c snack bar.
- Scenario 3:
  - Deposit 5 + 10 + 5c, take 20c snack bar.

PETRINET MODEL

EXAMPLE-2 VENDING MACHINE



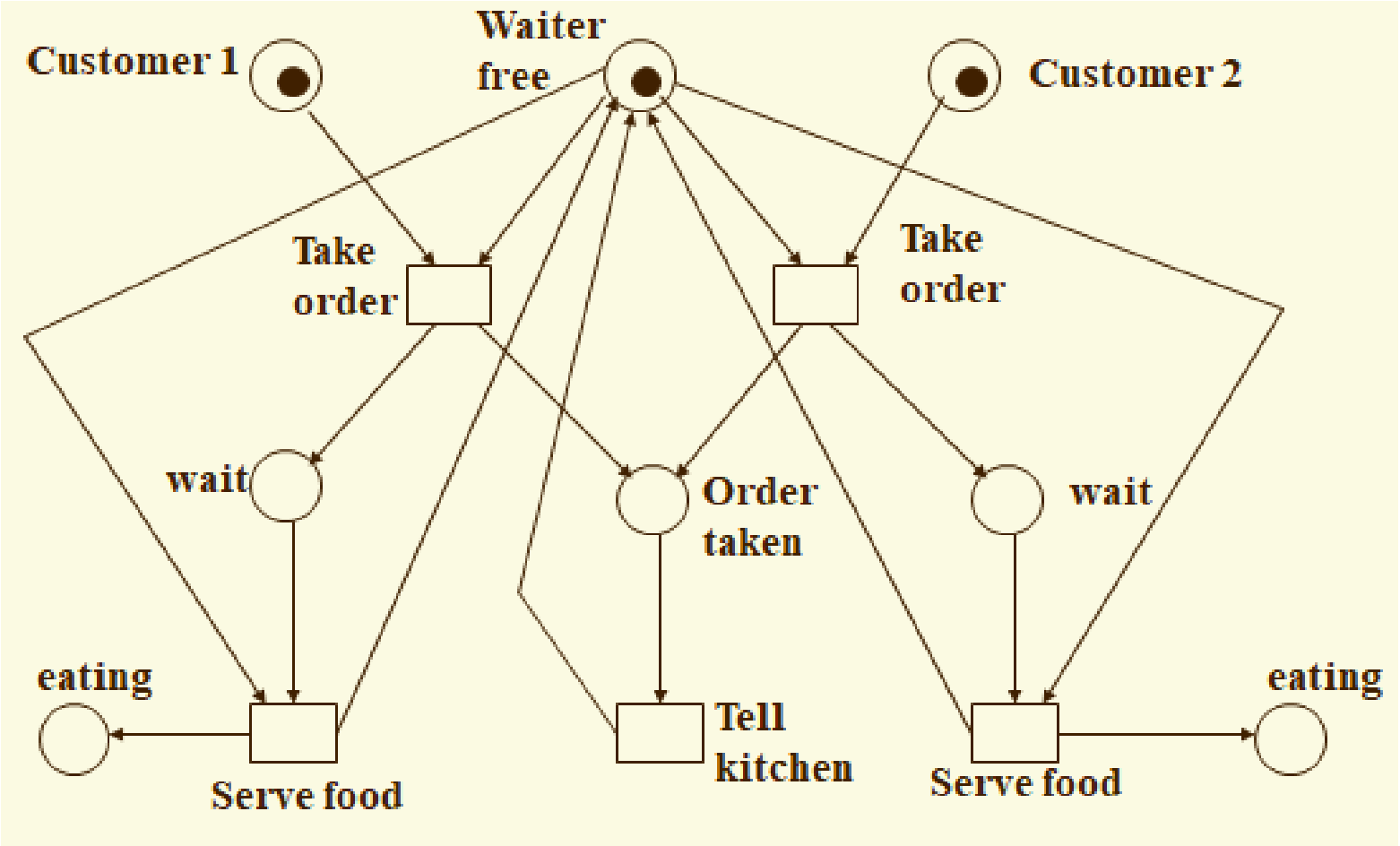
PETRINET MODEL

EXAMPLE-3 ORDER MANAGEMENT IN A RESTAURANT

- In the real world, many events are concurrent in nature. Many applications have global state formed from local state.
- Scenario 1:
  - Waiter takes order from customer 1
  - Serves customer 1
  - Takes order from customer 2
  - Serves customer 2.
- Scenario 2:
  - Waiter takes order from customer 1
  - Takes order from customer 2
  - Serves customer 2
  - Serves customer 1

PETRINET MODEL

EXAMPLE-3 ORDER MANAGEMENT IN A RESTAURANT



UNIFIED Modelling LANGUAGE (UML)

UNIFIED Modelling LANGUAGE (UML)

- **What is Modelling Language?**

A Modelling language is a graphical/textual computer language explains design and construction of any model or structure following a set of guidelines.

- **Importance of Modelling** : Visualization, reduced complexity, documentation

- To **effectively model a system**, you need a language with which the model can be described and here's where UML comes in.

UNIFIED Modelling LANGUAGE (UML)

INTRODUCTION

- UML - general purpose visual Modelling language
- It is used to visualize, specify, construct, and document
- Using tools codes can be generated with UML diagrams in various languages
- It is a pictorial language
- It can be used for both Modelling the software system and non-software systems
- For example, modelling a process flow in automated systems, etc.

UNIFIED Modelling LANGUAGE (UML)

INTRODUCTION

- UML is used in many applications by many people like business users, common people to make the system simple, clear and understandable
- It is a simple modelling mechanism to model most of the practical systems
- It is a object-oriented analysis based method

UNIFIED Modelling LANGUAGE (UML)

INTRODUCTION

UML Uses:

- Forecast systems
- To estimate the reusability.
- Lower costs
- Plan and analyze system behavior
- Easier maintenance/modification

UNIFIED Modelling LANGUAGE (UML)

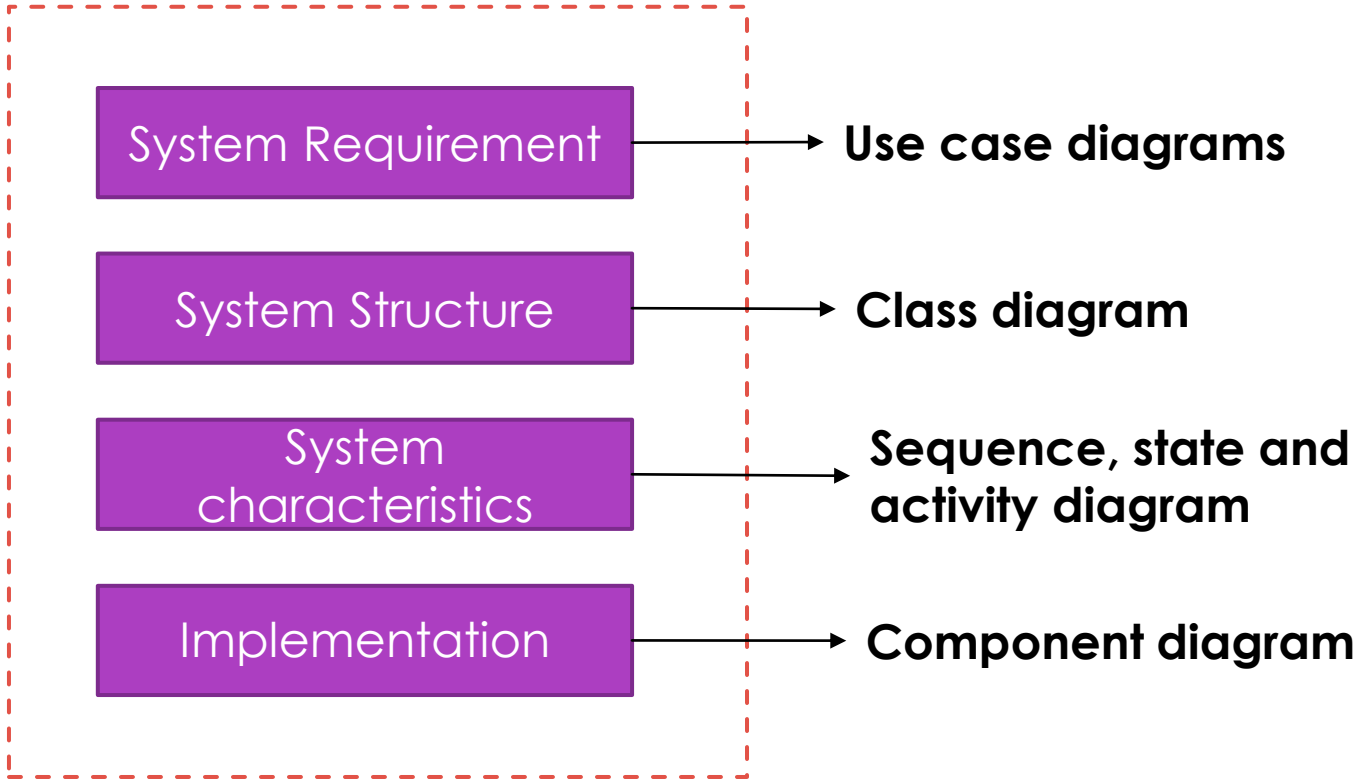
INTRODUCTION

There are two types of diagram in UML

1. Structure diagram: explains static structure of a system
   - Class diagram
   - Object diagram
   - Deployment diagram
   - Package diagram
2. Behavior diagram : used to model dynamic changes
   - Use case diagram
   - Interaction diagram
   - Activity diagram
   - State diagram

UNIFIED Modelling LANGUAGE (UML)

❑ Principles of UML Modelling



USE CASE DIAGRAM

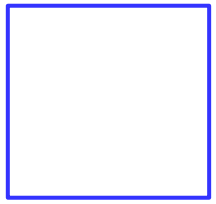
UNIFIED Modelling LANGUAGE (UML)

USE CASE DIAGRAM

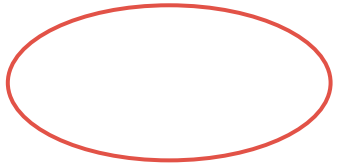
- The use case diagram shows different ways a user can interact with the system
- It can be used in scenarios like when system interact with people or any other external systems
- It contains “use-cases” and ‘’actors’’
- Use case diagram depicts how the actor interacts with use cases
- It describes the relationship between functionalities and their controllers

UNIFIED Modelling LANGUAGE (UML)

USE CASE DIAGRAM

System 

Actors 

Use Cases 

Relationships

UNIFIED Modelling LANGUAGE (UML)

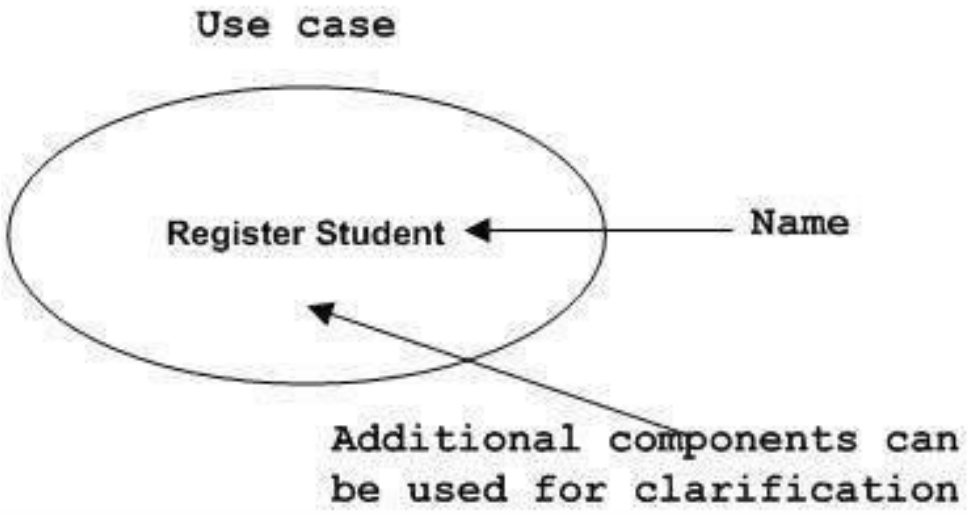
UML NOTATIONS

- Notations are very important in any modelling language
- Efficient use of notations play a major role in making any model meaningful
- Use case diagram: Notations of things and relationships
- Extensibility makes UML more powerful and flexible.

UNIFIED Modelling LANGUAGE (UML)

USE CASE NOTATIONS

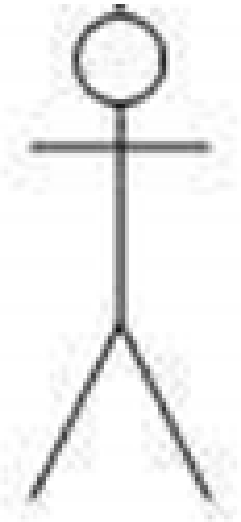
- Use case - high level functionalities of a system.
- Use case is represented as an eclipse



UNIFIED Modelling LANGUAGE (UML)

USE CASE NOTATIONS - ACTOR

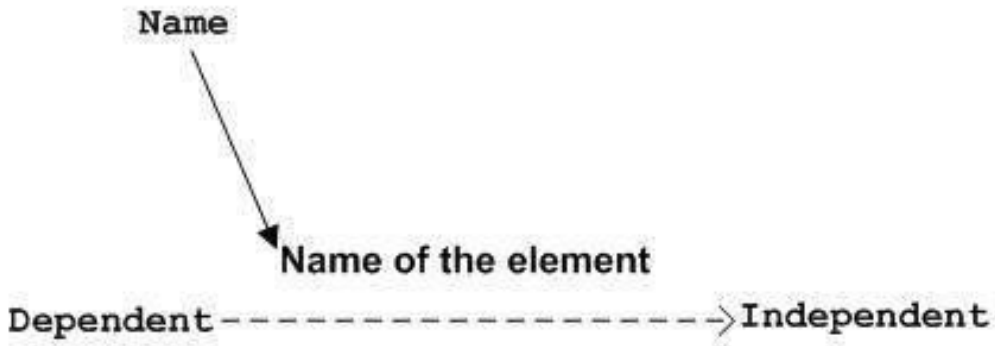
- An actor - the internal or external entities.



UNIFIED Modelling LANGUAGE (UML)

USE CASE NOTATIONS - DEPENDENCY

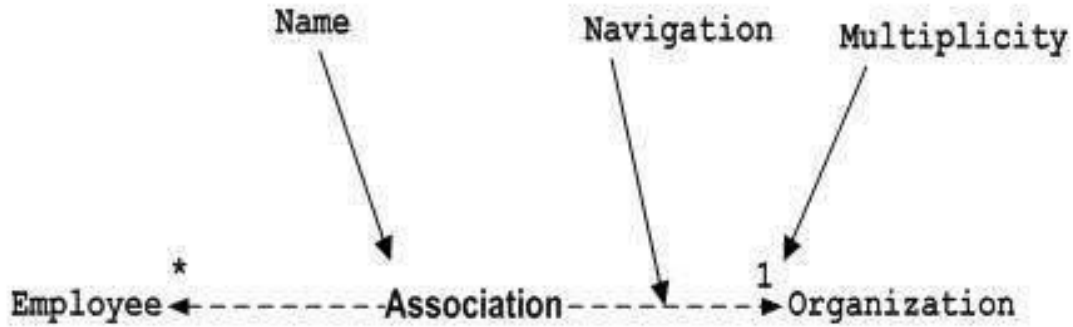
- Dependency - relationship between two elements of a system
- Dependency - dotted arrow
- Arrow head represents - independent element
- Other end - dependent element



UNIFIED Modelling LANGUAGE (UML)

USE CASE NOTATIONS - ASSOCIATION

- Association - relationship between two elements
- Association describes how the elements are associated.
- Association is represented by a dotted line with (without) arrows on both sides
- The multiplicity (1, *, etc.) to show how many objects are associated



UNIFIED Modelling LANGUAGE (UML)

USE CASE NOTATIONS - GENERALIZATION

- Generalization - parent-child relationship
- Generalization - inheritance relationship of the object-oriented concept
- Generalization - arrow with a hollow arrow head

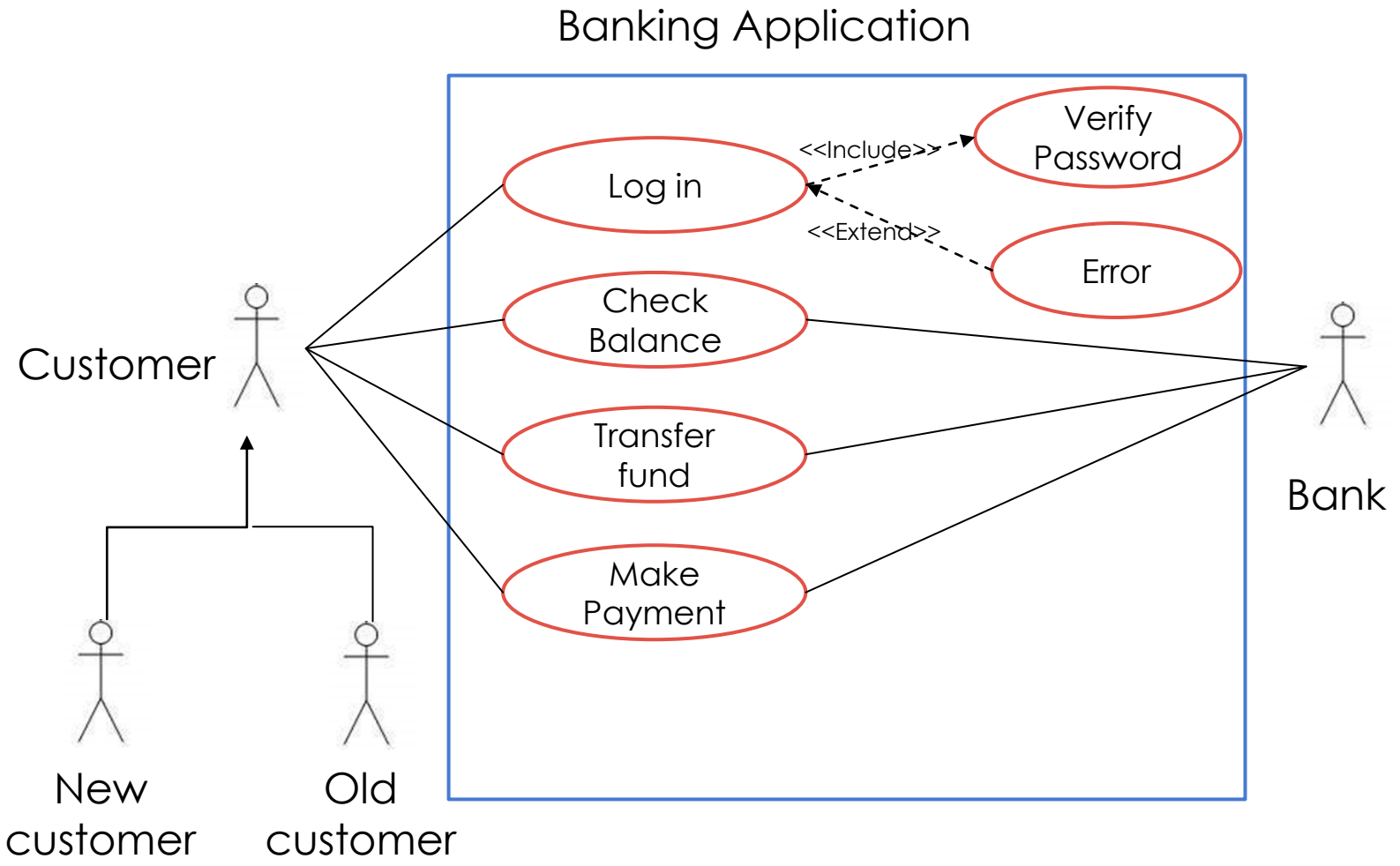


UNIFIED Modelling LANGUAGE (UML)

USE CASE NOTATIONS - <<include>> and <<extend>>

- ‘Extend’ and ‘Include’ – between use cases
- Include: invocation of one use case by the other
- Extend: extending use case will work exactly like the base use case

UNIFIED Modelling LANGUAGE (UML)



Primary actor – Initiates the process

Secondary actor – Responds

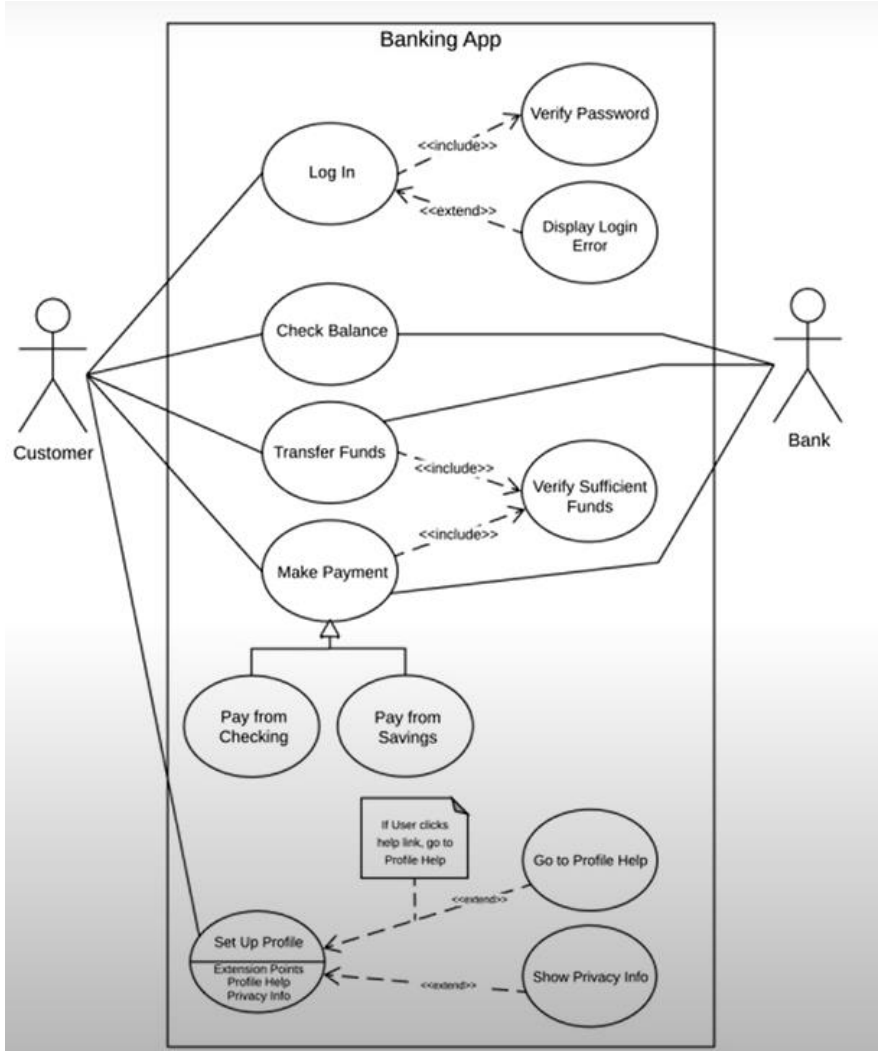
Use case – Action

Association

Exclude
Include

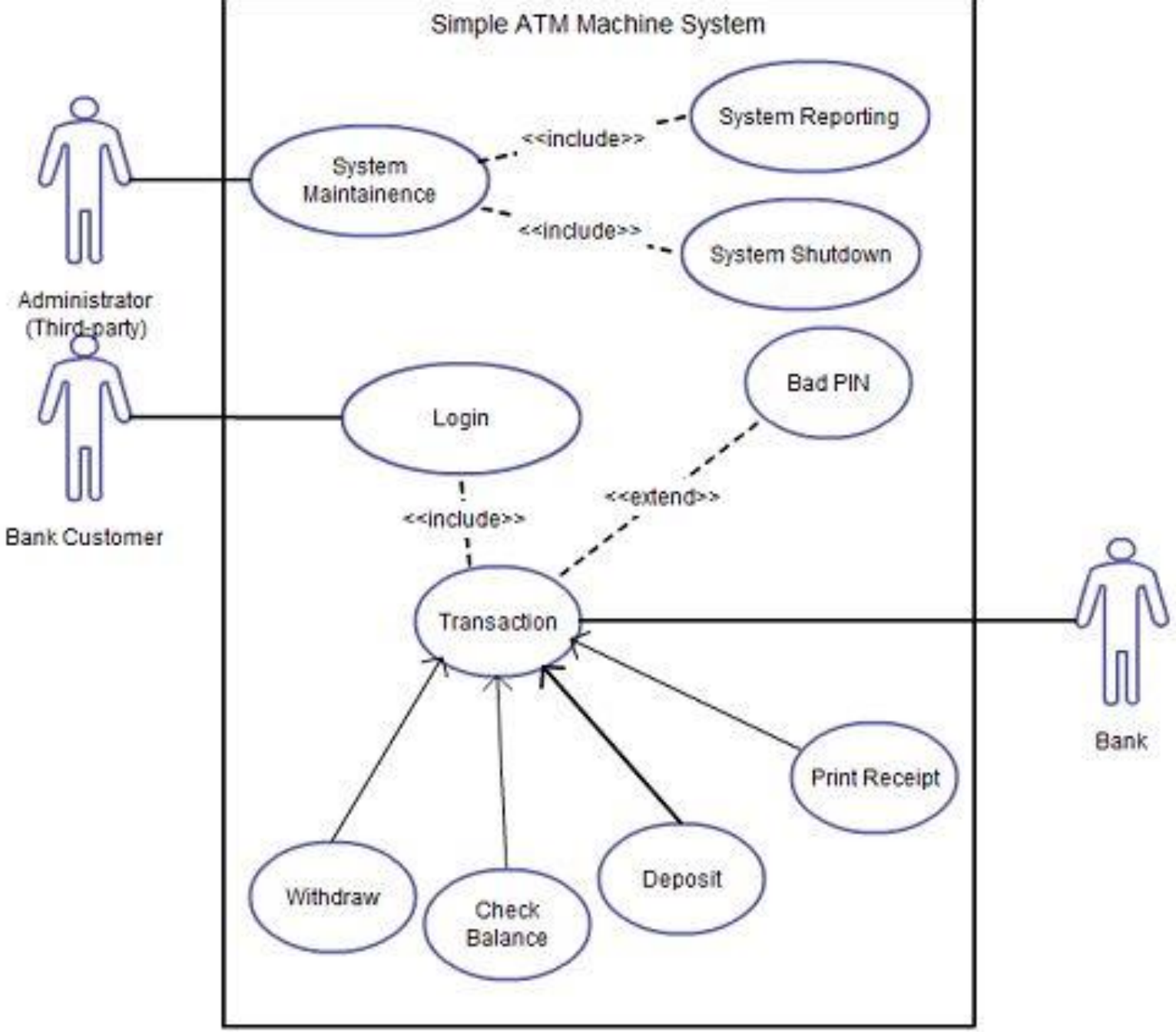
Generalization

UNIFIED Modelling LANGUAGE (UML)



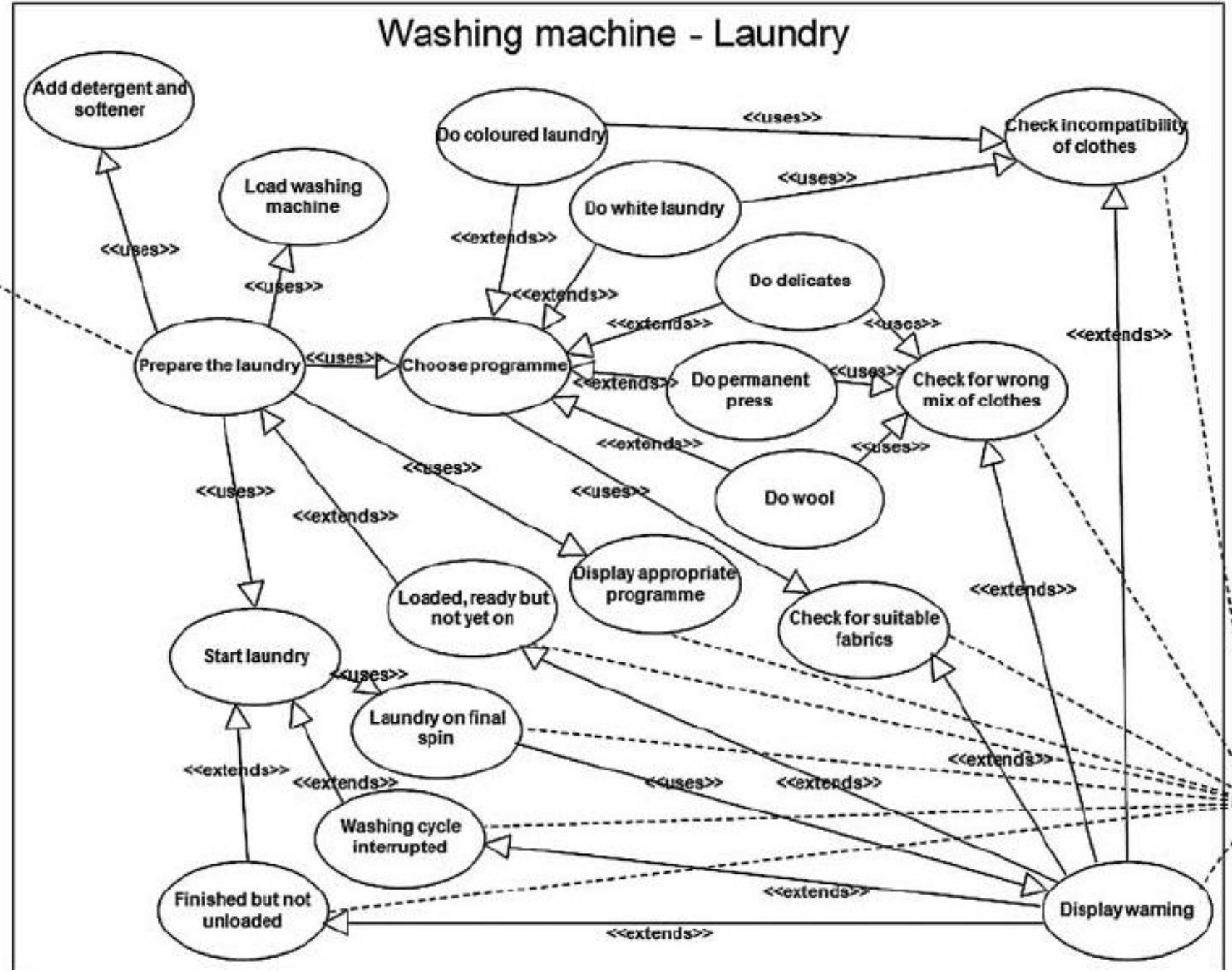
(UML)

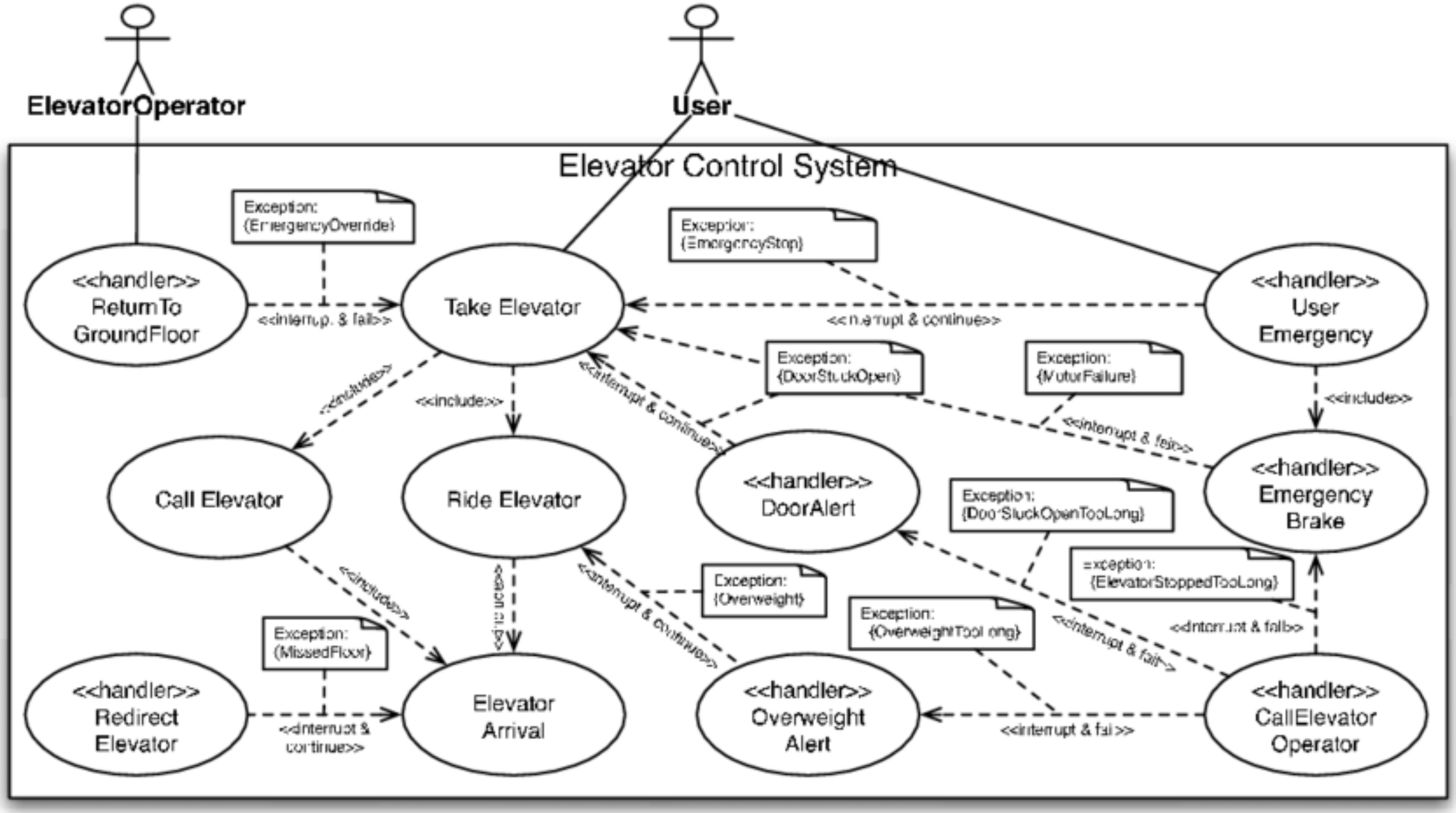
YSTEM


Simple ATM Machine System
System Reporting
<<include>>
System Maintainence
<<include>>
System Shutdown
Administrator (Third-party)
Bad PIN
Login
<<extend>>
<<include>>
Bank Customer
Transaction
Bank
Print Receipt
Withdraw
Check Balance
Deposit

UML)

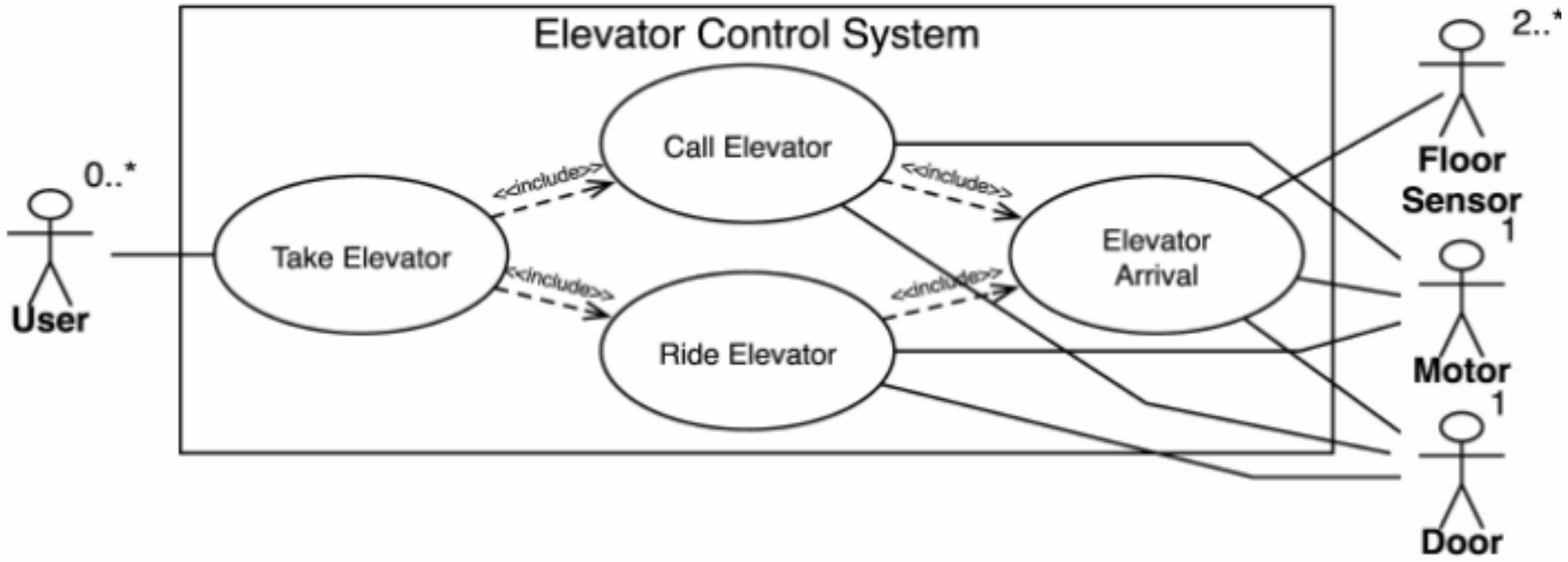
NE LAUNDRY




ElevatorOperator
User
Elevator Control System
Exception: (EmergencyOverride)
Exception: (EmergencyStop)
<<handler>> ReturnTo GroundFloor
<<interrupt & fail>>
Take Elevator
<<interrupt & continue>>
<<handler>> User Emergency
Exception: (DoorStuckOpen)
Exception: (MotorFailure)
<<include>>
<<include>>
<<include>>
<<interrupt & continue>>
<<interrupt & fail>>
Call Elevator
Ride Elevator
<<handler>> DoorAlert
Exception: (DoorStuckOpenTooLong)
<<handler>> Emergency Brake
<<include>>
<<include>>
<<include>>
Exception: (Overweight)
Exception: (ElevatorStoppedTooLong)
Exception: (MissedFloor)
Exception: (OverweightTooLong)
<<interrupt & continue>>
<<interrupt & fail>>
<<interrupt & fail>>
<<handler>> Redirect Elevator
<<interrupt & continue>>
Elevator Arrival
<<handler>> Overweight Alert
<<interrupt & fail>>
<<handler>> CallElevator Operator

UNIFIED Modelling LANGUAGE (UML)

USE CASE DIAGRAM EXAMPLE ELEVATOR SYSTEM



CLASS DIAGRAM

UNIFIED Modelling LANGUAGE (UML)

CLASS DIAGRAM

- Class diagram- use to document software architecture
- It is used to refine the use case diagram
- Set of interrelated classes
- It contains methods and attributes for classes
- Classes can be "is-a" or "has-a" relationship

UNIFIED Modelling LANGUAGE (UML)

CLASS DIAGRAM

- Class diagram - Static diagram, Structural diagram
- Class diagram - attributes and operations of a class
- It explains the constraints on the system
- It is a collection of classes, interfaces, associations, collaborations and constraints

UNIFIED Modelling LANGUAGE (UML)

CLASS DIAGRAM

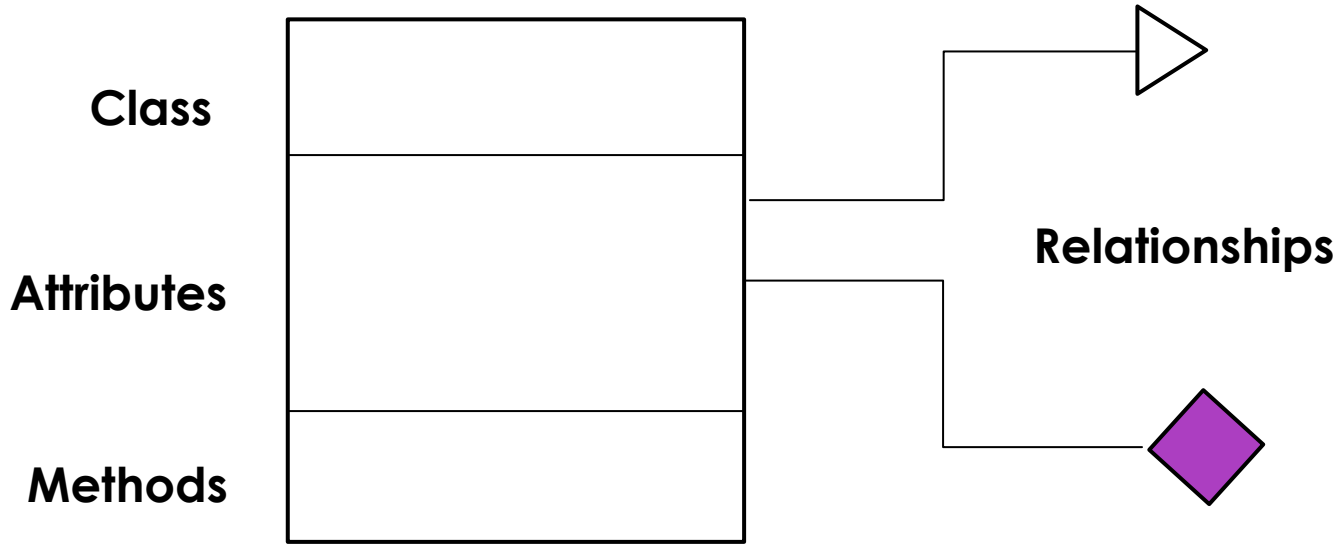
- The purpose of the class diagram can be summarized as –
  - Analysis and design of system
  - Describe responsibilities of a system
  - Conceptual modelling
  - Forward and reverse engineering

UNIFIED Modelling LANGUAGE (UML)

UML NOTATIONS - CLASS

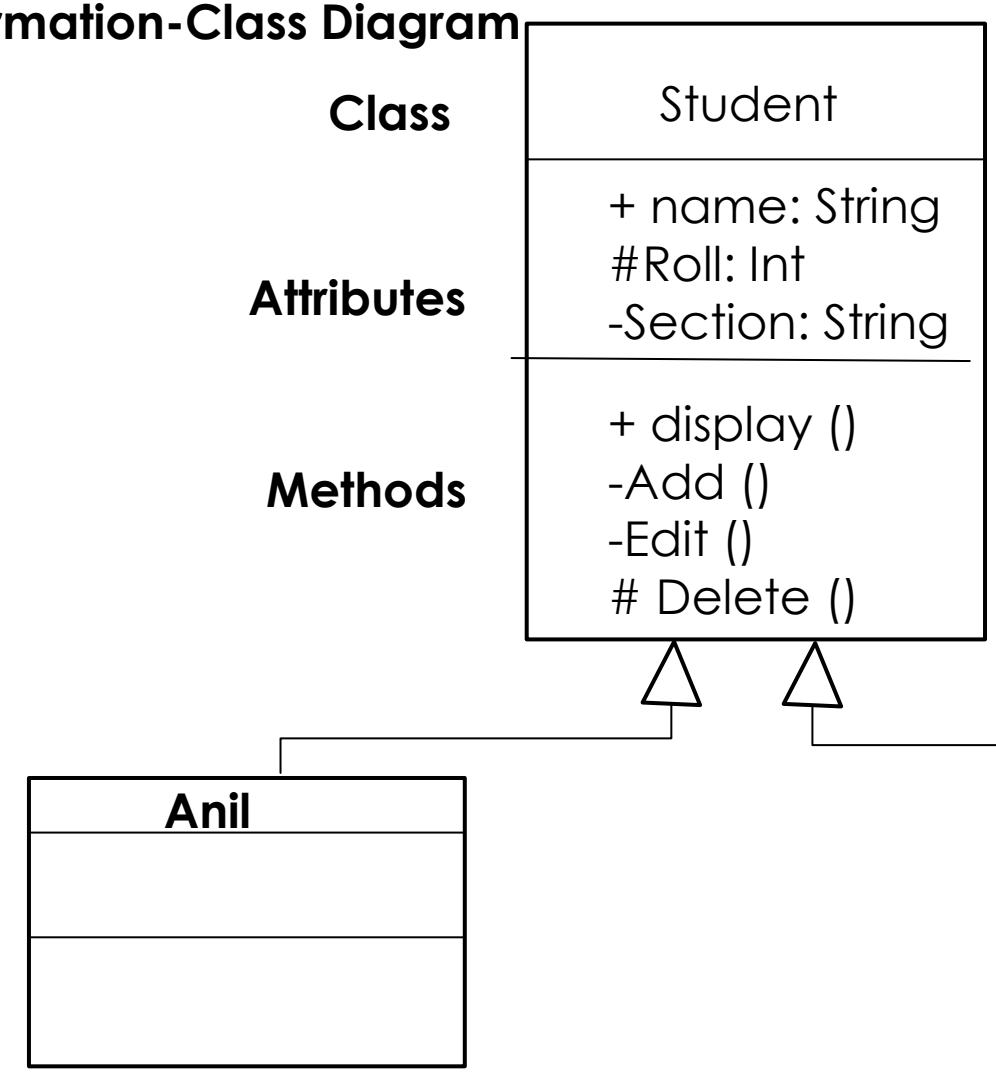
- Classes are used to represent objects
- UML class diagram is divided into four parts.
  - Top section - name the class.
  - Second - attributes of the class.
  - Third - operations or methods performed by the class.
  - Fourth - any additional components.

UNIFIED Modelling LANGUAGE (UML)



UNIFIED Modelling LANGUAGE (UML)

Student Information-Class Diagram



Visibility
+ Public
- Private
protected
~ Package/default

Inheritance

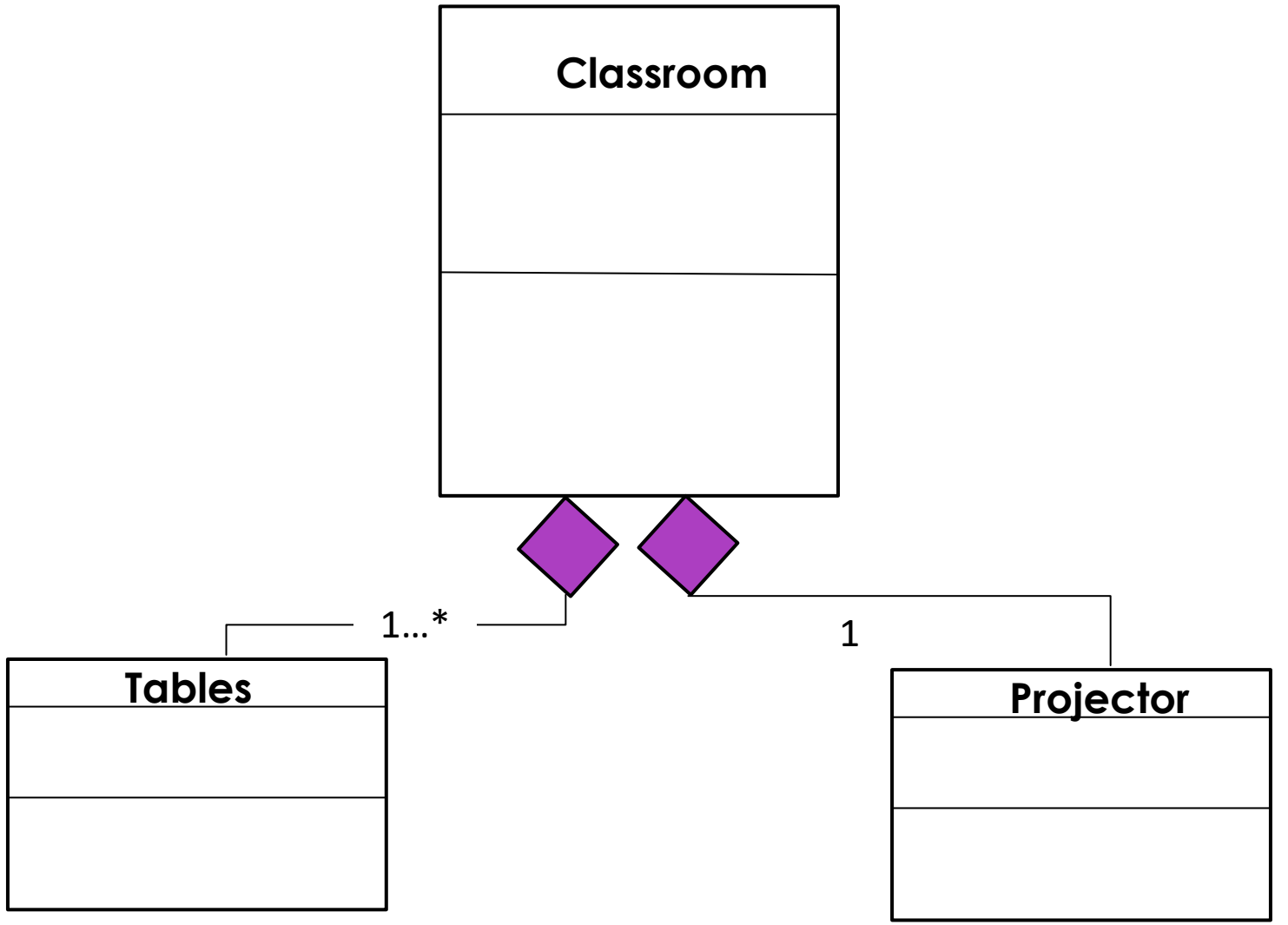


Association

Aggregation



UNIFIED Modelling LANGUAGE (UML)



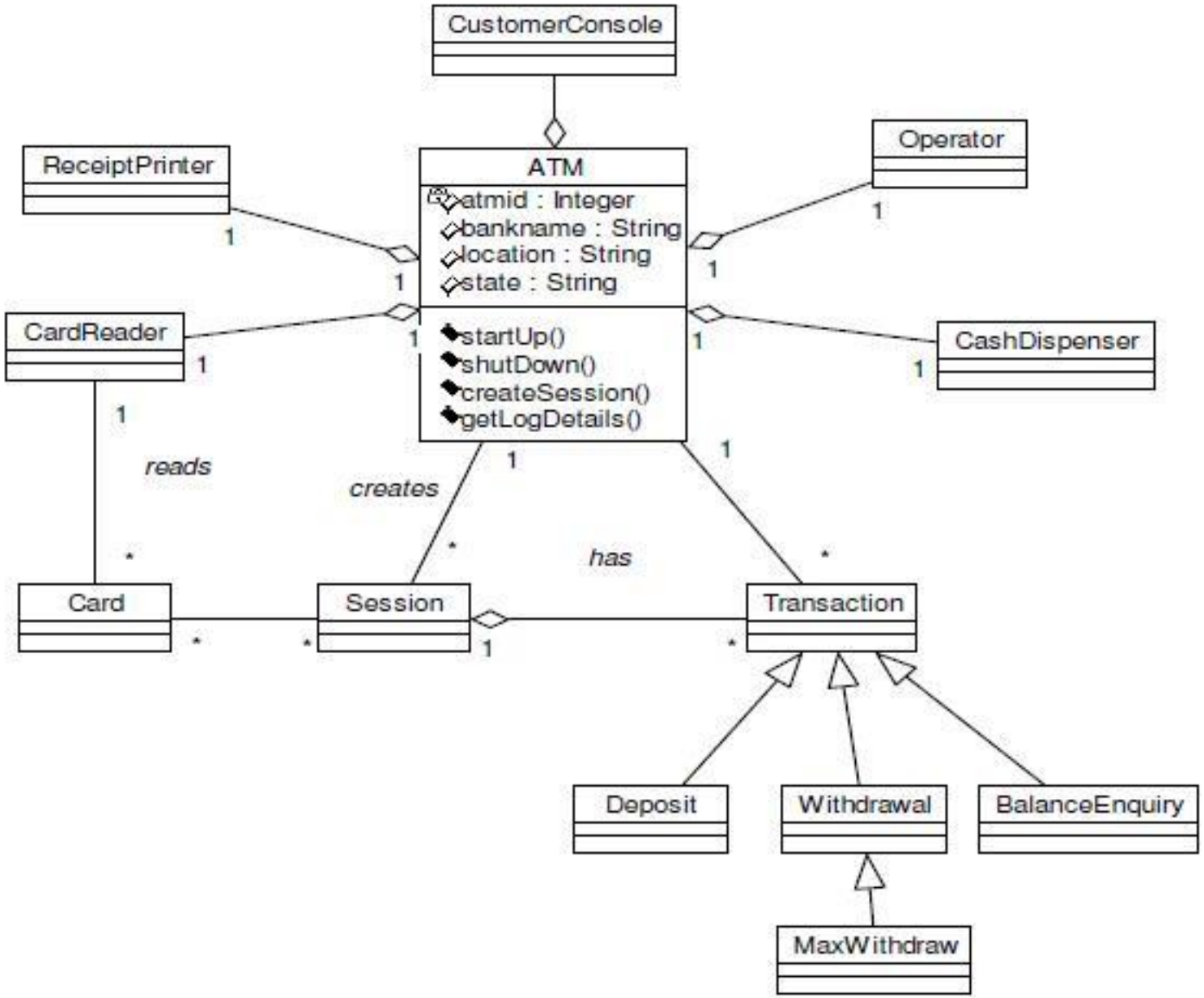
Multiplicity

N
0...*
0...1
1...*
m...n

Composition

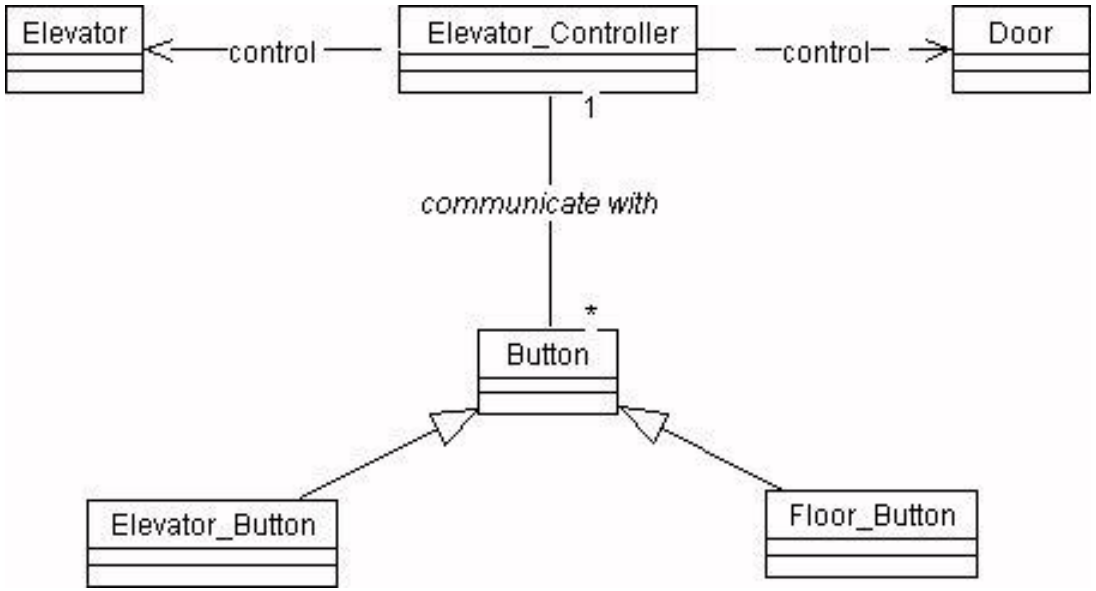


UNIFIED Modelling LANGUAGE (UML)

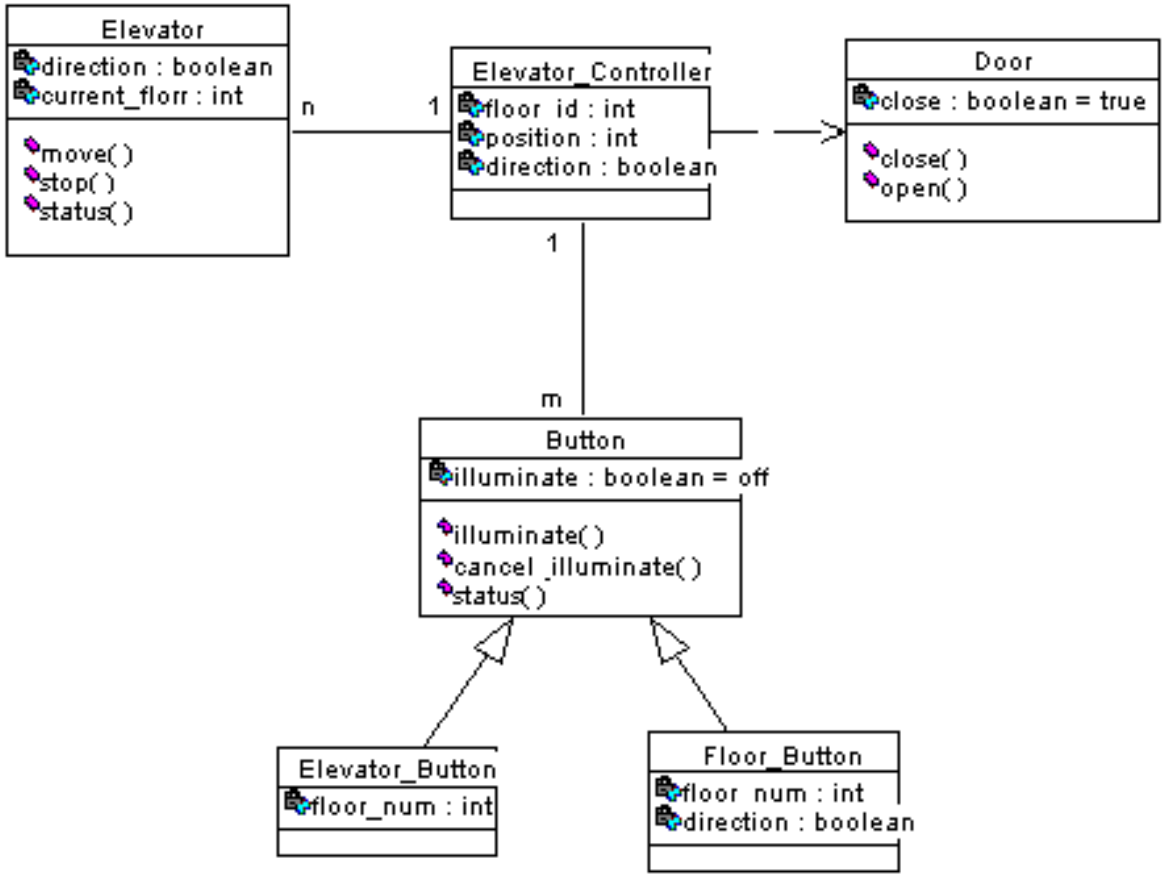


UNIFIED Modelling LANGUAGE (UML)

CLASS DIAGRAM EXAMPLE ELEVATOR SYSTEM



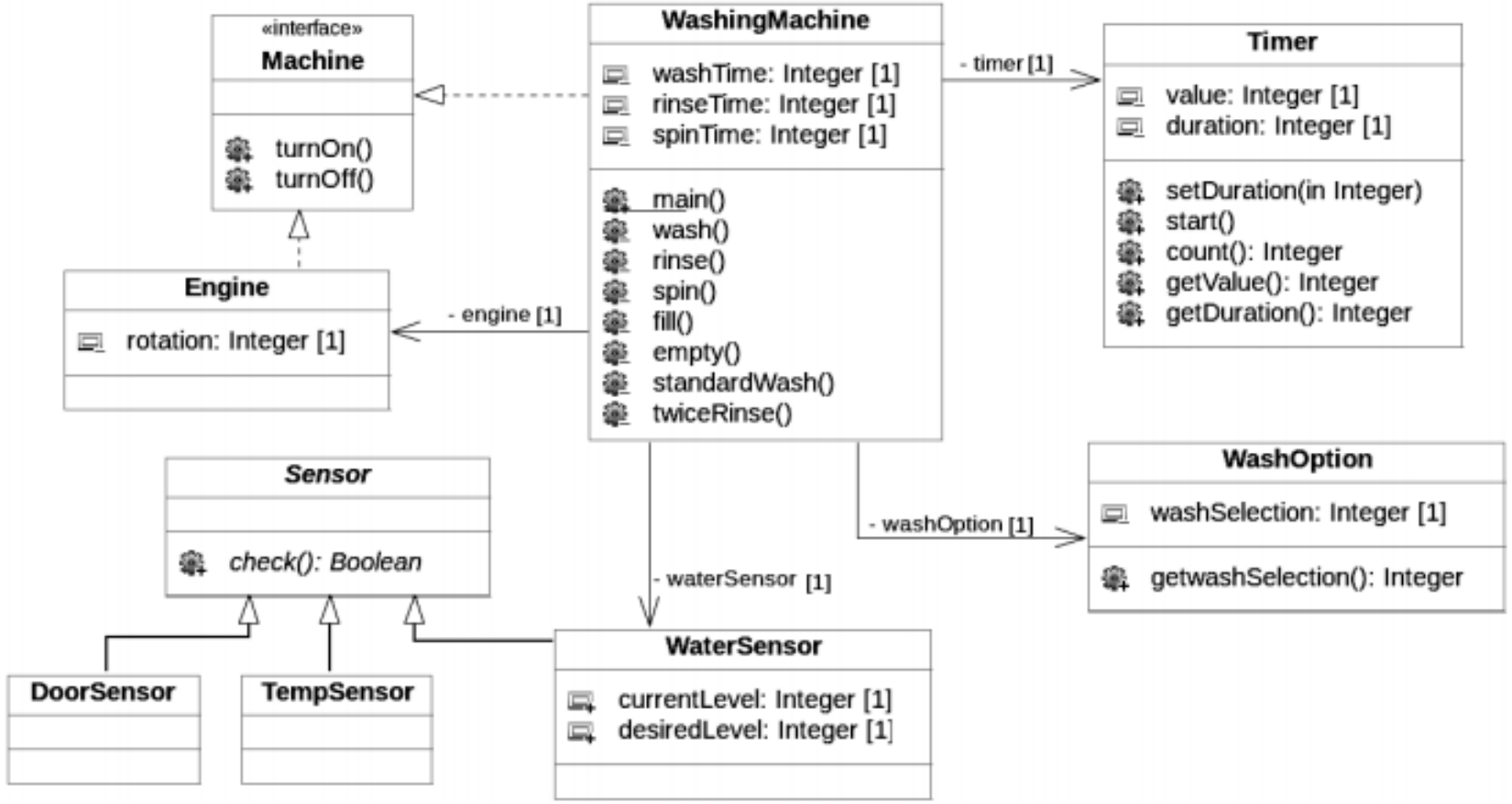
Simplified Class Diagram



Detailed Class Diagram

UNIFIED Modelling LANGUAGE (UML)

CLASS DIAGRAM EXAMPLE WASHING MACHINE



SEQUENCE DIAGRAM

UNIFIED Modelling LANGUAGE (UML)

SEQUENCE DIAGRAM

- Sequence diagram - interactions among the different elements in the model
- It describes the dynamic behavior of the system
- They explain the interaction between objects and collaborations
- It emphasizes on the time sequence of messages and structural organizations of the objects

UNIFIED Modelling LANGUAGE (UML)

SEQUENCE DIAGRAM

- Sequence diagram shows the details of the use cases
- It shows the complete flow of information, function and operations of the system
- It is used in planning and understanding the functionality of existing and future scenario
- Time – vertical direction
- Header elements – Horizontal direction

UNIFIED Modelling LANGUAGE (UML)

SEQUENCE DIAGRAM

- Sequence Diagrams are suitable for following scenarios:
  - Usage scenario – to determine how the system could be used
  - Method logic – to explore logic of any function, procedure or complex process
  - Service logic – ideal way to explain high level methods used by clients

UNIFIED Modelling LANGUAGE (UML)

Actor



Object

Lifeline Notation

Activation Bar

Synchronous message

Return message

Comment

Alternate frame



UNIFIED Modelling LANGUAGE (UML)

SEQUENCE DIAGRAM EXAMPLE PRODUCT PURCHASE SYSTEM

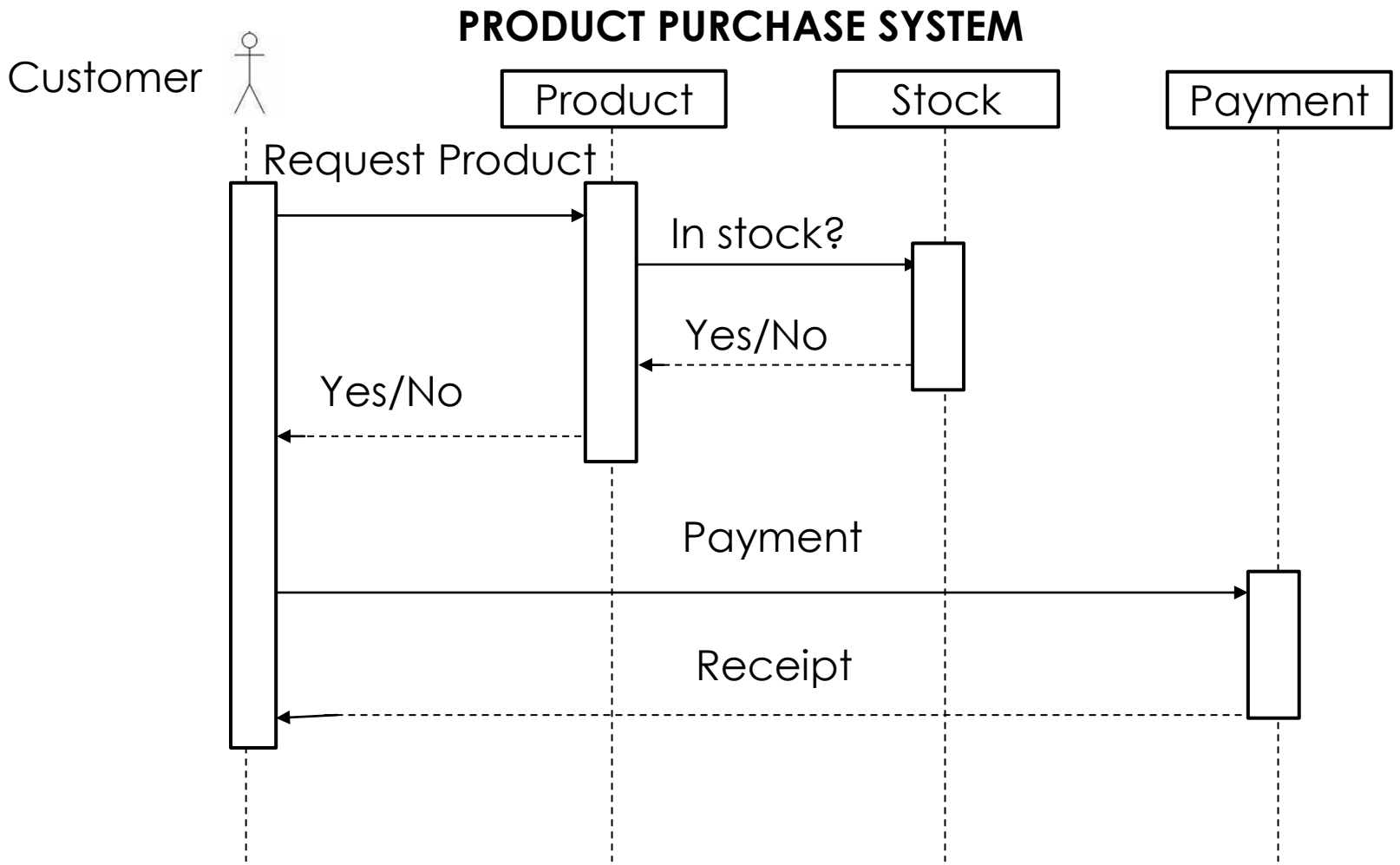
- In this sequence diagram we have four objects
  - Customer
  - Product
  - Stock
  - Payment
- The message starts from the top and flows to the bottom (waterfall manner)
- Dashed lines - duration for object
- Horizontal rectangles - activation of the object
- Messages sent - dark arrow and dark arrow head
- Return message - dotted arrow

UNIFIED Modelling LANGUAGE (UML)

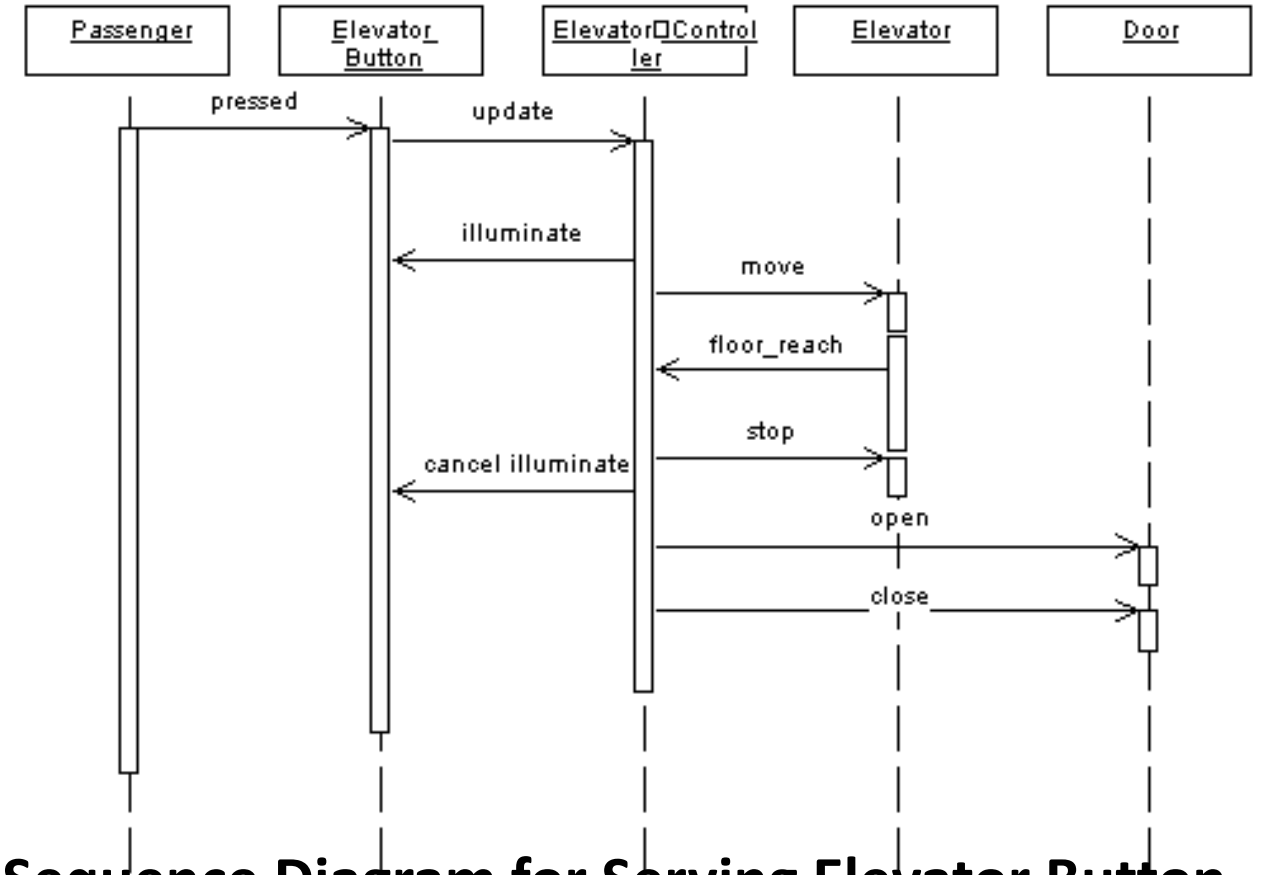
SEQUENCE DIAGRAM EXAMPLE - PRODUCT PURCHASE SYSTEM

- Customer object - to the product object
- Product object -to the stock object
- Stock object - saying yes or No.
- Product object sends - customer object.
- Customer object -payment object to pay money.
- Payment object - receipt to the customer object.

UNIFIED Modelling LANGUAGE (UML)

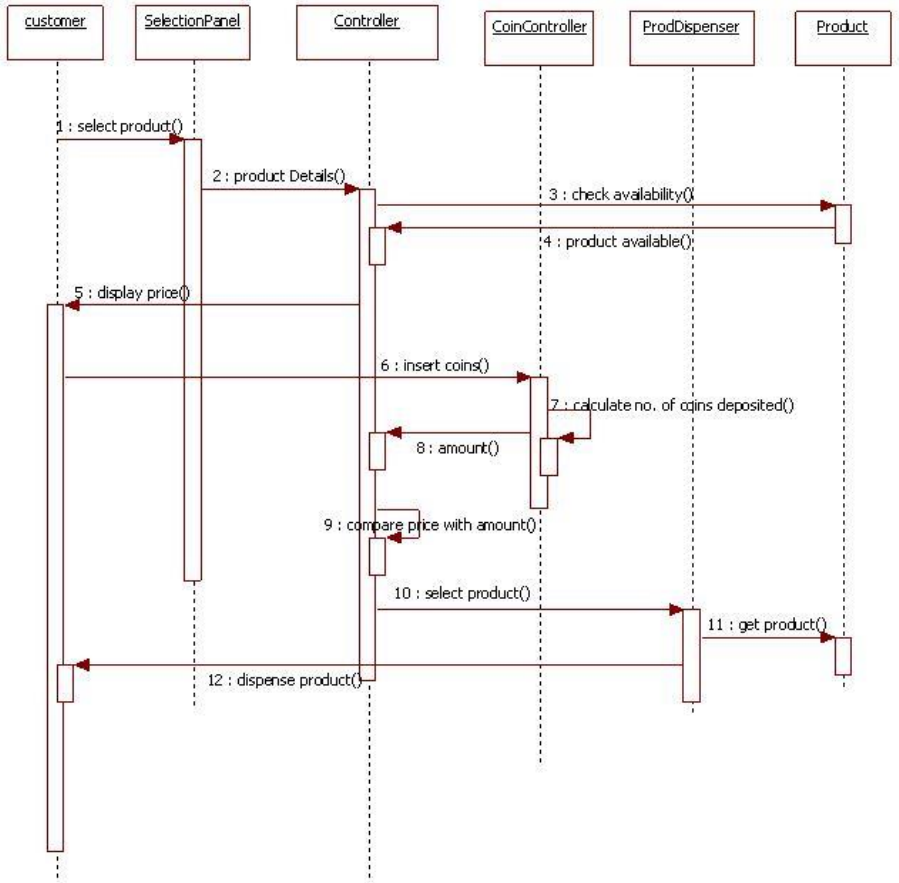


SEQUENCE DIAGRAM EXAMPLE ELEVATOR SYSTEM



Sequence Diagram for Serving Elevator Button

SEQUENCE DIAGRAM EXAMPLE VENDING MACHINE



STATE DIAGRAM

UNIFIED Modelling LANGUAGE (UML)

STATE DIAGRAM

- State diagram stores the status of object
- It shows the condition of system over a finite instance of time
- It's a behavioral diagram
- It also referred to as State machines and State-chart Diagrams.
- It explains the dynamic behavior of a class
- State diagrams an be used to understand the reaction of objects on receiving external stimuli

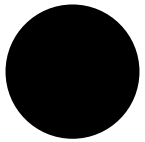
UNIFIED Modelling LANGUAGE (UML)

STATE DIAGRAM

- **Uses of state chart diagram:**
  - We use it to depict event driven objects
  - We use it for showing use cases in business context
  - Shows the overall behavior of state machine
  - We use it to model the dynamic behavior of the system .
- **Steps to draw a state diagram:**
  - Identify the initial state and the final states.
  - Identify the possible states in which the object can exist
  - Label the events which trigger these transitions.

UNIFIED Modelling LANGUAGE (UML)

STATE DIAGRAM SYMBOL NOTATIONS



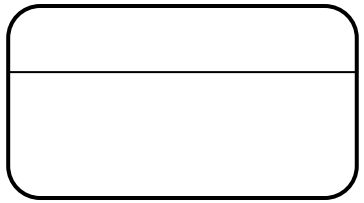
Initial state



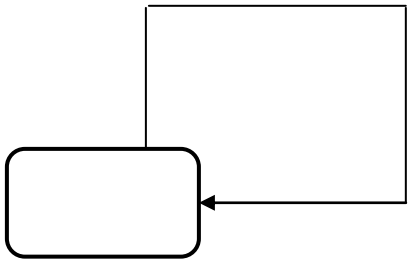
Transition Symbol



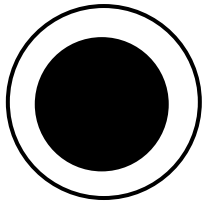
State



Composite State



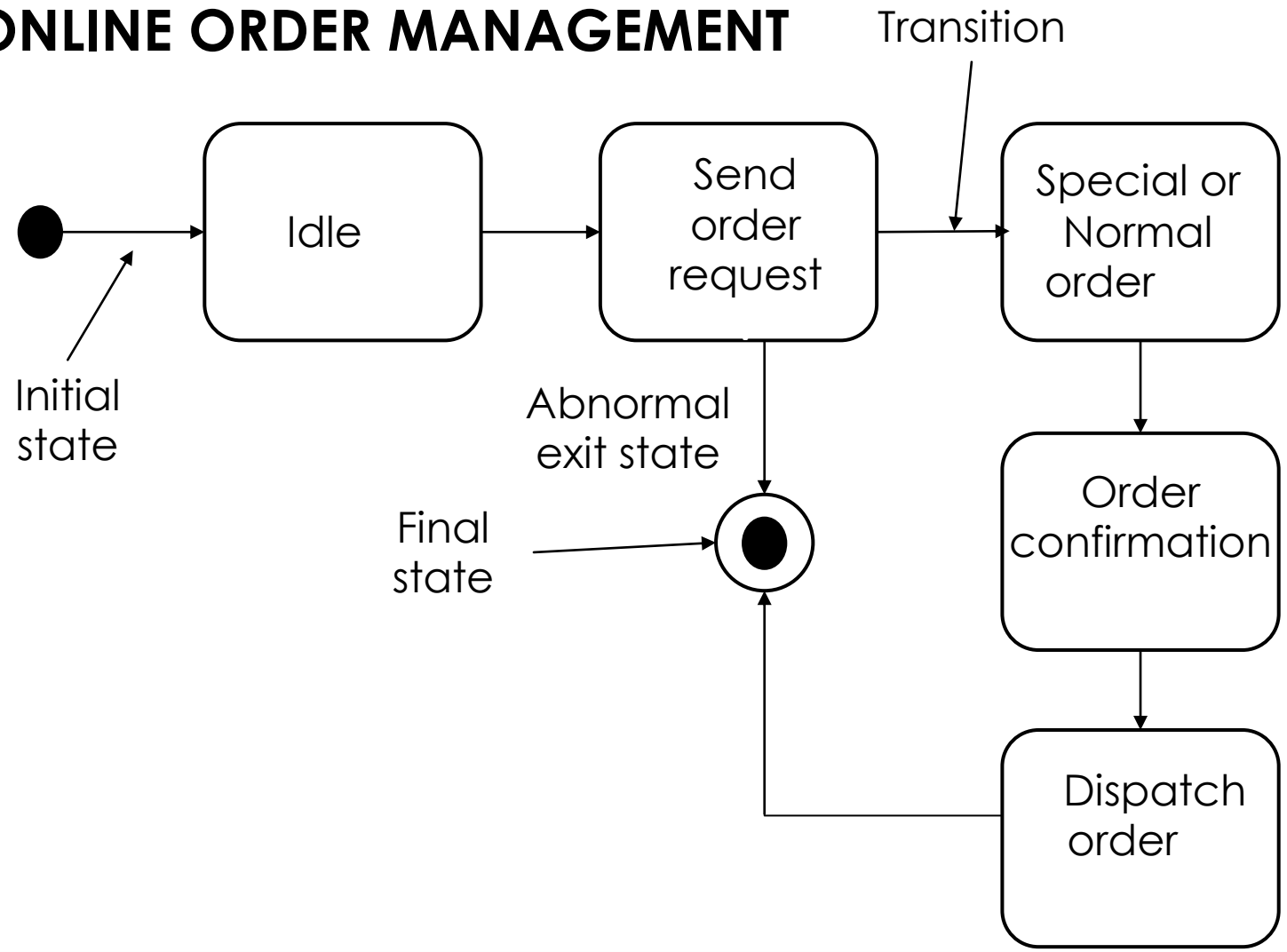
Self Transition



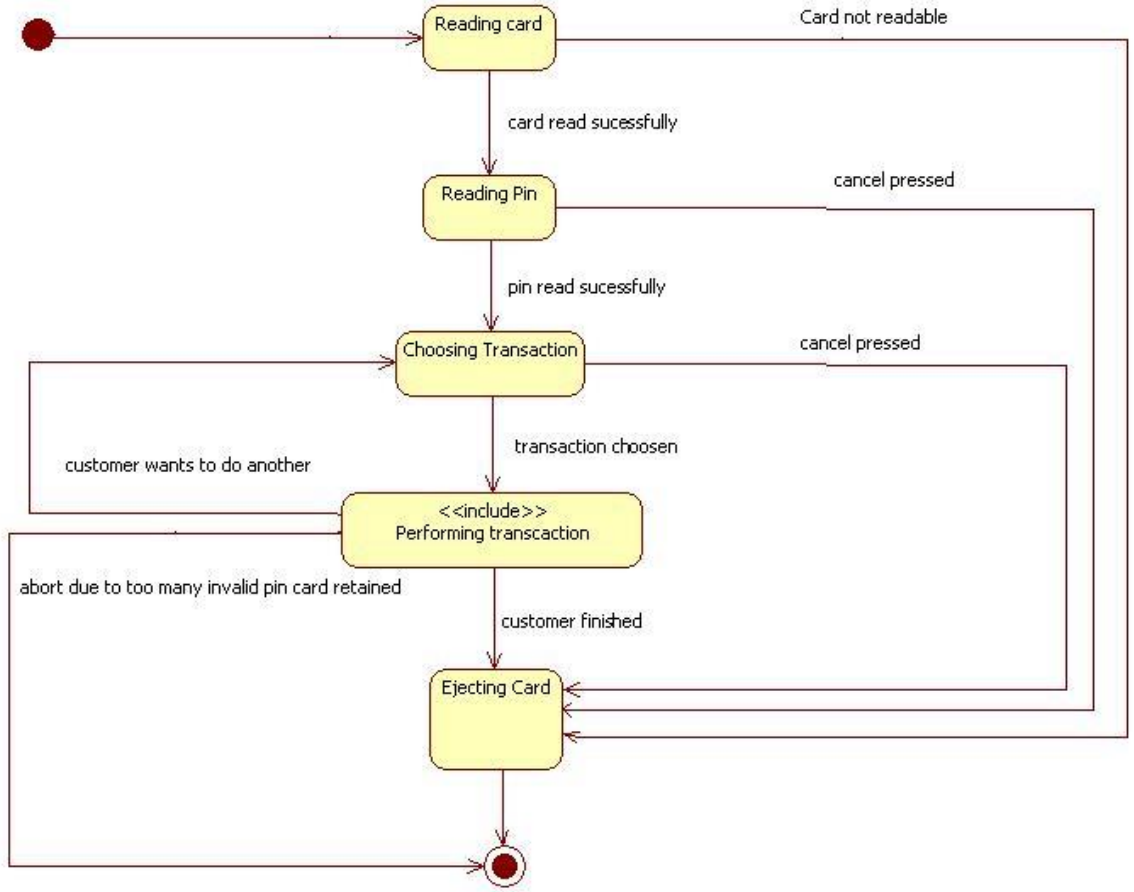
Final state

UNIFIED Modelling LANGUAGE (UML)

ONLINE ORDER MANAGEMENT

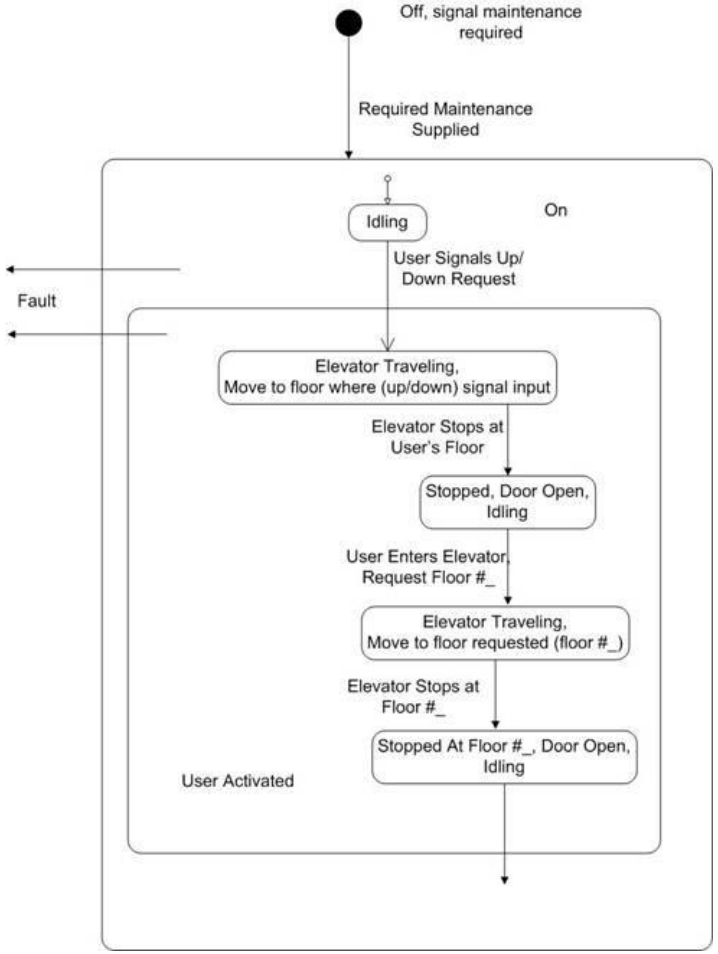


STATE DIAGRAM EXAMPLE ATM MACHINE



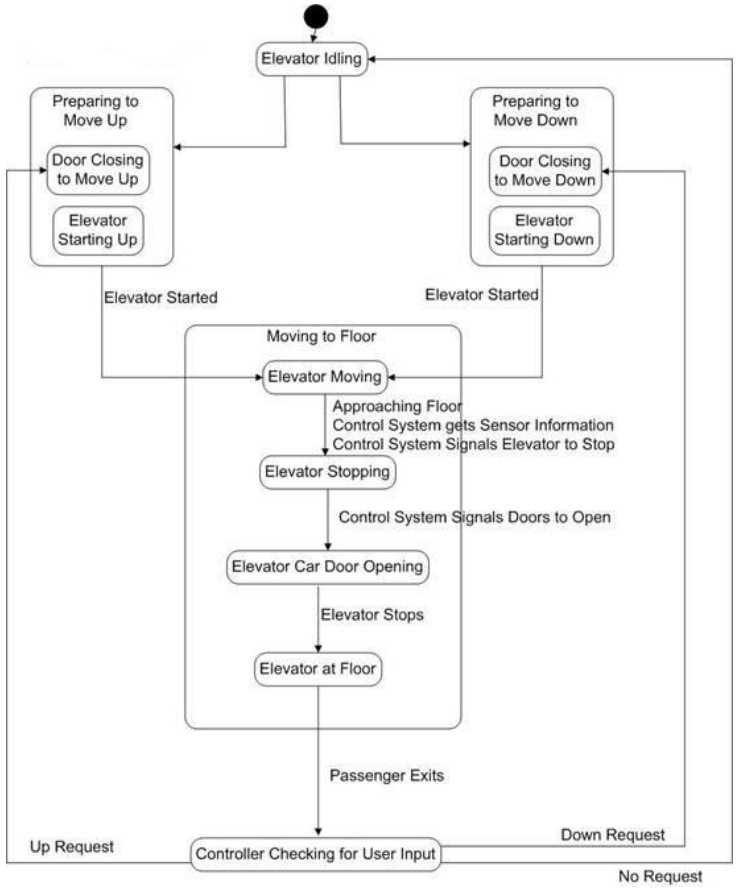
UNIFIED Modelling LANGUAGE (UML)

STATE DIAGRAM EXAMPLE ELEVATOR SYSTEM



Statechart for "User" Behavior

Statechart for "Elevator Control System"



ACTIVITY DIAGRAM

UNIFIED Modelling LANGUAGE (UML)

ACTIVITY DIAGRAM

- An activity diagram - behavioral diagram
- It shows the control flow from start to finish point including other decisions paths that is being executed
- It can explain both sequential and concurrent processing

UNIFIED Modelling LANGUAGE (UML)

ACTIVITY DIAGRAM

- **Uses of Activity chart diagram:**
  - Modelling work flow by using activities
  - Modelling the logic of algorithm
  - Shows the workflow process between users and system
  - Understanding system's functionalities.
  - Investigating business requirements at a later stage.

UNIFIED Modelling LANGUAGE (UML)

ACTIVITY DIAGRAM

- **Steps to draw a activity diagram:**
  - Identify the initial state and the final states.
  - Identify the intermediate activities needed to reach the final state from he initial state.
  - Identify the conditions or constraints which cause the system to change control flow.
  - Draw the diagram with appropriate notations.

UNIFIED Modelling LANGUAGE (UML)

ACTIVITY DIAGRAM SYMBOL NOTATIONS



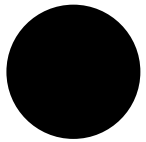
Activity



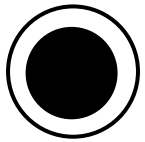
Action

Control flow

Object flow



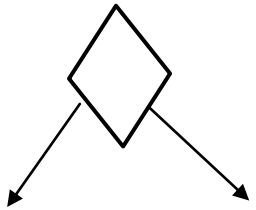
Initial node



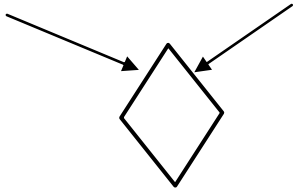
Final node



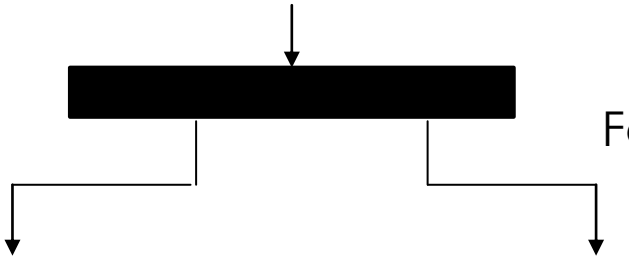
Object node



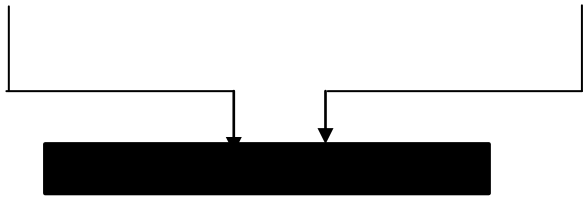
Decision node



Merge node



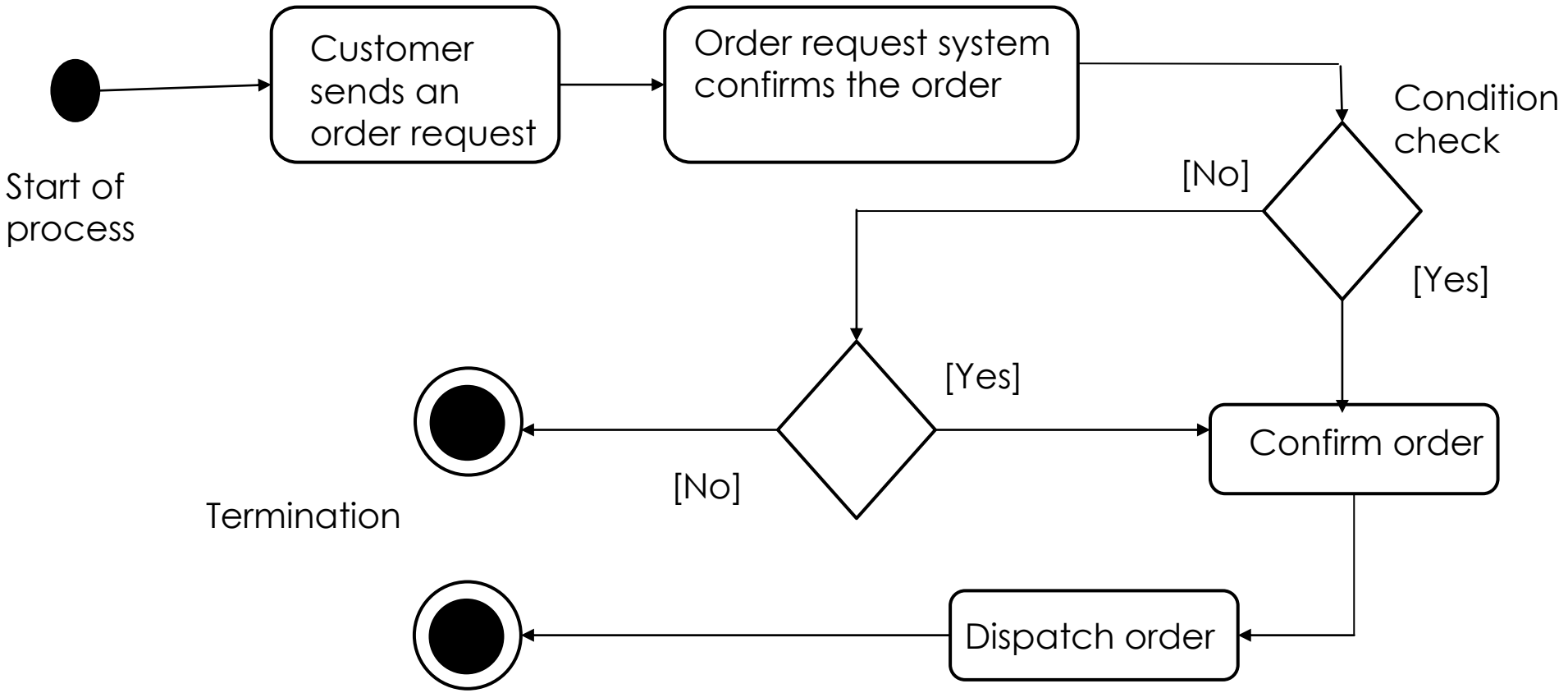
Fork node



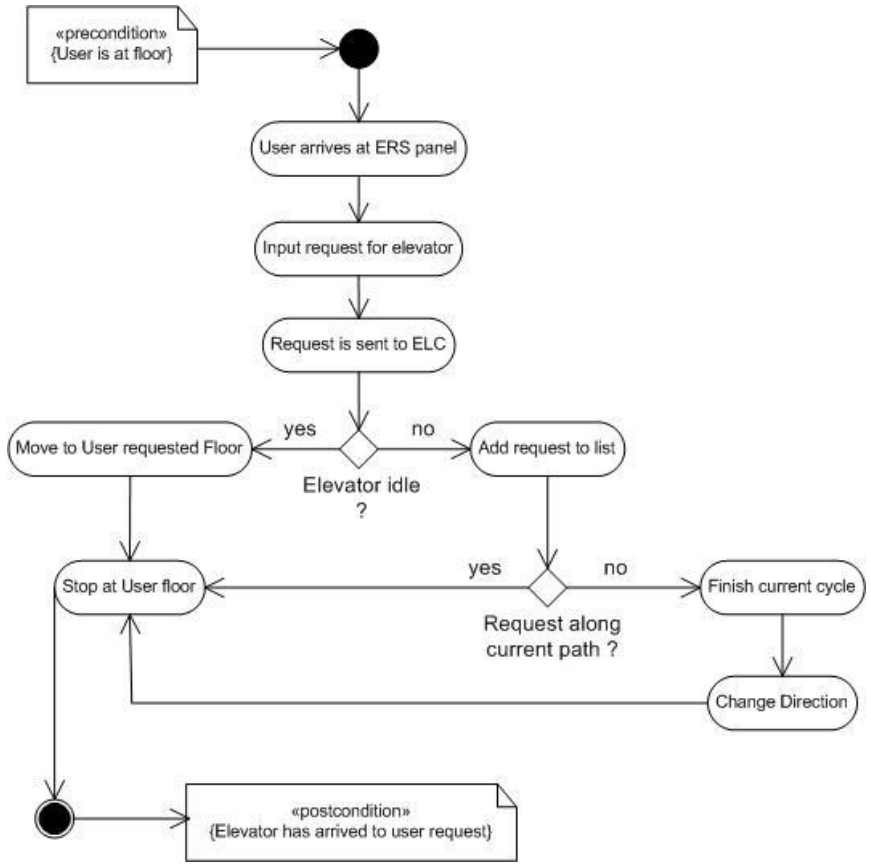
Join node

UNIFIED Modelling LANGUAGE (UML)

Activity Diagram of an Order Management System



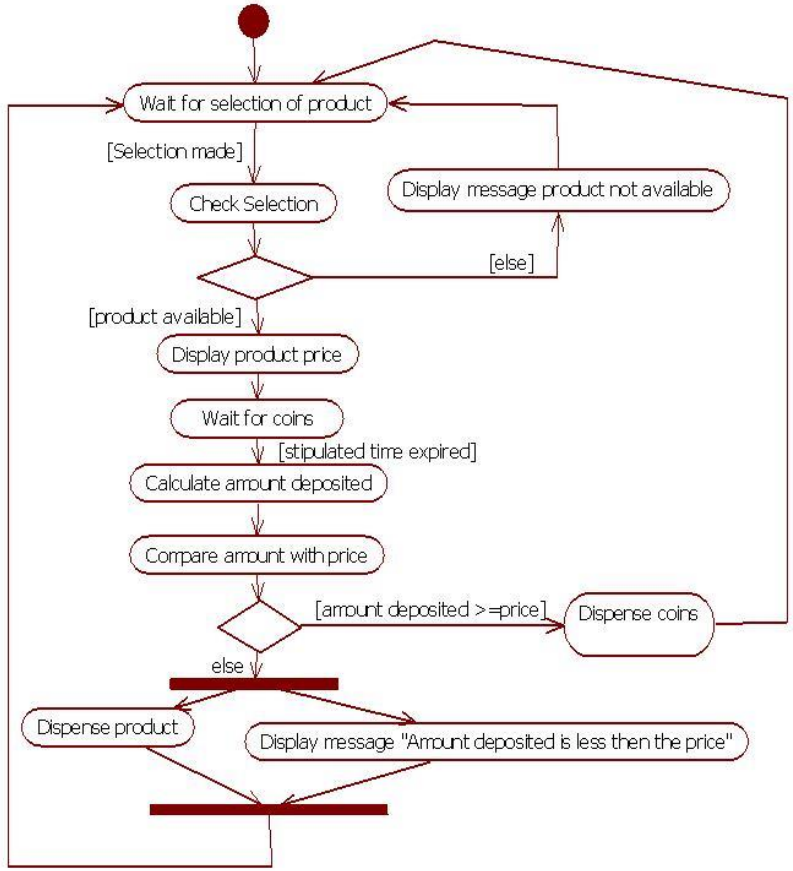
ACTIVITY DIAGRAM EXAMPLE ELEVATOR SYSTEM



Request Elevator Activity

ELC – Elevator Logic Control
ERS – Elevator Request Service

ACTIVITY DIAGRAM EXAMPLE VENDING MACHINE



Code Optimization

Optimization

- In computing, optimization is the process of modifying a system to make some aspect of it work more efficiently or use fewer resources.
- Ex: a computer program may be optimized so that it executes more rapidly or is capable of operating with less memory storage or other resources, or draw less power.
- The system may be a single computer program, a collection of computers or even an entire network such as the internet.

Code Optimization

- ***Optimization*** is a program transformation technique, which tries to improve the code that consume less resources (i.e. CPU, Memory) and deliver high speed.
- In optimization, high-level general programming constructs are replaced by very efficient low-level programming codes.
- Usually done at the end of the development stage since it reduces readability & adds code that is used to improve the performance.

Code Optimization

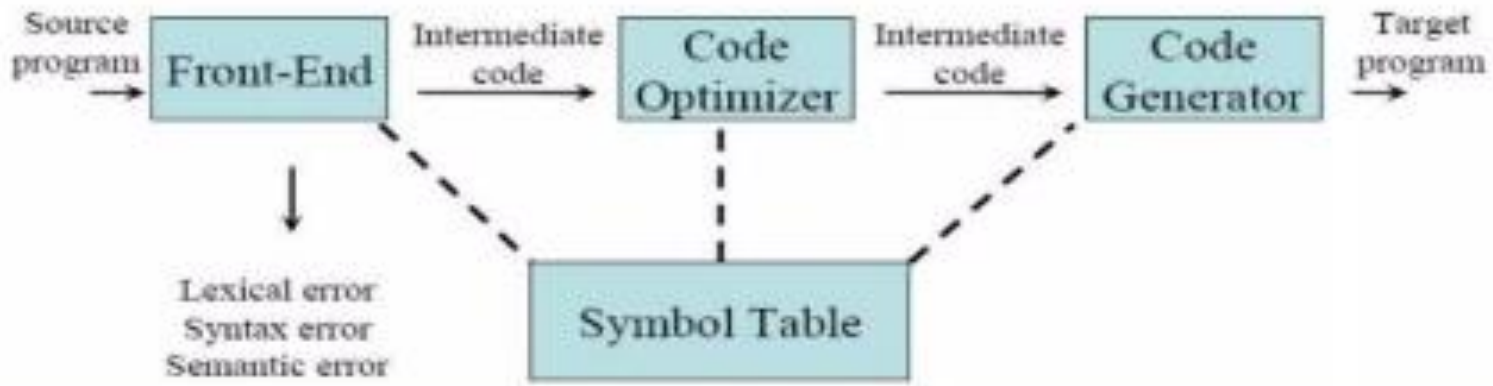
Optimized code's features:-

- Executes faster
- Code size get reduced
- Efficient memory usage
- Yielding better performance
- Reduces the time and space complexity

Code Optimization

Optimizations are classified into two categories:

- ***Machine independent optimizations*** - improve target code without taking properties of target machine into consideration.
- ***Machine dependent optimization*** - improve target code by checking properties of target machine.



Code Optimization

***Criteria for Optimization*:-**

- An optimization must preserve the meaning of a program:

  -Cannot change the output produced for any input.

  -Can not introduce an error.
- Optimization should, on average, speed up programs.
- Optimization should itself be fast and should not delay the overall compiling process.

Code Optimization

Optimization can occur at several levels:

1. ***Design level:-*** At the highest level, the design may be optimized to make best use of the available resources.
   - The implementation of this design will benefit from the use of suitable efficient algorithms and the implementation of these algorithms will benefit from writing good quality code.
2. ***Compile level:-*** Use of an optimizing compiler tends to ensure that the executable program is optimized at least as much as the compiler can predict

Code Optimization

Improvements can be made at various phases:-

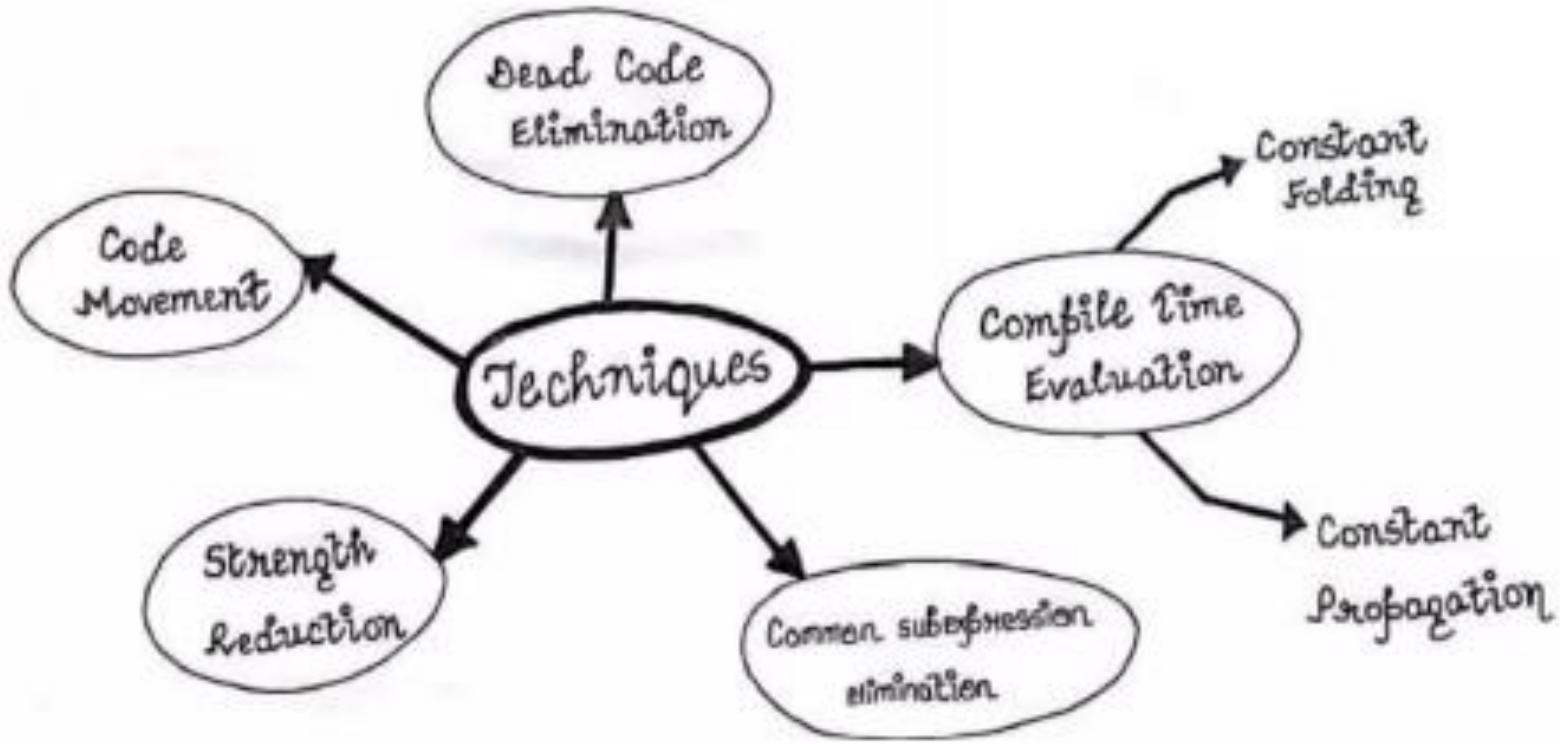
- ***Source Code***: Algorithm's transformation can produce spectacular improvements
- ***Intermediate Code***: Compiler can improve loops, procedure calls and address calculations.

  -Typically only optimizing compilers include this phase

- ***Target Code***: Compilers can use registers efficiently

Code Optimization

- Main techniques of Optimization:-



Code Optimization

There are primarily 3 types of optimizations:-

(1) Local optimization - Apply to a basic block in isolation

(2) Global optimization - Apply across basic blocks

(3) Loop optimization – Apply on loops

Another significant optimization can be:-

(4) peep-hole optimization - Apply across boundaries

Code Optimization

- Optimization performed within a basic block.
- This is simplest form of optimizations
- No need to analyze the whole procedure body.
- Just analyze the basic blocks of the procedure.
- The local optimization techniques include:
  - Constant Folding
  - Constant Propagation
  - Algebraic Simplification
  - Operator Strength Reduction
  - Dead Code Elimination

Local Code Optimization

A) Constant folding-

- As the name suggests, this technique involves folding the constants by evaluating the expressions that involves the operands having constant values at the compile time.
- Ex: Circumference of circle = (22/7) x Diameter
- Replace (22/7) with 3.14 during compile time and save the execution time.

Local Code Optimization

B) Constant Propagation-

- In this technique, if some variable has been assigned some constant value, then it replaces that variable with its constant value in the further program wherever it has been used during compilation, provided that its value does not get alter in between.
- Ex: pi = 3.14, radius = 10
- Area of circle = pi x radius x radius
- Substitutes constants value during compile time and saves the execution time.

Local Code Optimization

C) Algebraic Simplification-

- Use algebraic properties to simplify expressions
- Some expressions can be simplified by replacing them with an equivalent expression that is more efficient.
- Ex: `-(-i)` → `i`

```
void f (int i)
{
 a[0] = i + 0;
 a[1] = i * 0;
 a[2] = i - i;
 a[3] = 1 + i + 1;
}
```



```
void f (int i)
{
 a[0] = i;
 a[1] = 0;
 a[2] = 0;
 a[3] = 2 + i;
}
```

Code Optimization

D) Strength Reduction-

- As the name suggests, this technique involves reducing the strength of the expressions by replacing the expensive and costly operators with the simple and cheaper ones

Code before Optimization	Code after Optimization
B = A x 2	B = A + A

Here, the expression "A x 2" has been replaced with the expression "A + A" because the cost of multiplication operator is higher than the cost of addition operator.

Code Optimization

E) Dead code elimination-

- Those code are eliminated which either never executes or are not reachable or even if they get execute, their output is never utilized.

Code before Optimization	Code after Optimization

i = 0 ;
if (i == 1)
{
a = x + 5 ;
}
	i = 0 ;

Code Optimization

F) Common sub-expressions elimination:

- Eliminates the redundant expressions and avoids their computation again and again.

Code before Optimization	Code after Optimization
S1 = 4 x i S2 = a[S1] S3 = 4 x j S4 = 4 x i **// Redundant Expression** S5 = n S6 = b[S4] + S5	S1 = 4 x i S2 = a[S1] S3 = 4 x j S5 = n S6 = b[S1] + S5

Global Code Optimization

- Optimization across basic blocks within a procedure /function
- Could be restricted to a smaller scope, Example: a loop
- Data-flow analysis is done to perform optimization across basic blocks
- Each basic block is a node in the flow graph of the program.
- These optimizations can be extended to an entire control - flow graph
- Most of compiler implement global optimizations with well founded theory and practical gains

Peephole Optimization

- Optimization technique that operates on the one or few instructions at a time.
- Performs machine dependent improvements
- Peeps into a single or sequence of two to three instructions (peephole) and replaces it by most efficient alternative (shorter or faster) instructions.
- Peephole is a small moving window on the target systems

Peephole Optimization

Characteristics of peep-hole optimizations:-

- Redundant-instruction (loads and stores)elimination
- Flow-of-control optimizations - - Elimination of multiple jumps (i.e. goto statements)
- Elimination of unreachable code
- Algebraic simplifications
- Reducing operator strength
- Use of machine idioms

replace **Add #1,R**
by **Inc R**

Loop Optimization

- Loop optimization plays an important role in improving the performance of the source code by reducing overheads associated with executing loops.
- Loop Optimization can be done by removing:
  - Loop invariant
  - Induction variables

Loop Optimization

- Removal of Loop Invariant data:-

```
i = 1
s= 0
do{
s= s + i
a =5
i = i + 1
{
while (i < =n)
```



```
i = 1
s= 0
a =5
do{
s= s + i
i = i + 1
{
while (i < =n)
```

Bringing a=5 outside the do while loop, is called code motion.

Loop Optimization

- Induction variables:-

```
i = 1
s= 0
S1=0
S2=0
while (i < =n)
{
s= s + a[ i ]
t1 = i * 4
s= s + b[ t1 ]
t2 = t1 +2
s2= s2 + c[ t2 ]
i = i + 1
}
```



```
i = 1
s= 0
S1=0
S2=0
t2=0
while (i < =n)
{
s= s + a[ i ]
t1 = t1+ 4
s= s + b[ t1 ]
s2= s2 + c[t1 +2 ]
i = i + 1
}
```

"+" replaced " * ", t1 was made independent of i

t1,t2 are induction variables. i is inducing t1 and t1 is inducing t2

Logic Analyzer

- A logic analyzer is an electronic measurement device that captures and displays multiple signals from a digital design
- It is an excellent tool for checking and debugging ICs, digital systems, circuits such as embedded systems, electronic control units, computers, etc.
- A modern logic analyzer can convert the captured data into timing diagrams, protocol decodes, state machine traces, and assembly language.

Logic Analyzer





Logic Analyzer

- The logic analyzer allows different types of standard interfaces to be automatically decoded.
- Depending on the analyzer these can include UART, SPI, I2C, CAN and many more.

Characteristics:

- *Multiple channels*
- *Provide a time display of logic states*
- *Displays logic states*
- *Does not display analog information*

Programming Environment

Embedded Software Development Tools:

- Editor
- Compiler
- Assembler
- Debugger
- Linker
- Emulator

Embedded Software Development Tools List

- PyCharm
- WebStorm
- Qt Creator
- MPLAB X
- Eclipse
- Visual Studio
- NetBeans
- MATLAB
- Arduino IDE
- ARM Keil

THANK YOU…